



Faculty of Computing and Informatics(FCI)

CSE6234
Software Design

Trimester March 2026, Term 2610

**Software Design Proposal & Software Design
Pattern Document**

E-Commerce-TechVault

By Group 1

ID	NAME
1211303991	Muhammad Ammar Ajwad
243UC246PD	Atodan Rizky Parijogo
243UC246LY	Sham Chalan

TABLE OF CONTENTS

Section	Title	Page
—	Abstract Summary	3
1	Introduction	4
1.1	Abstract	4
1.2	Problem Statement	4
1.3	Project Objectives	5
1.4	Literature Review	6
1.5	Project and System Scope	7
2	System Overview	10
2.1	Generic Use Case	10
2.2	Class Diagrams	13
2.3	Entity Relationship Diagram (ERD)	13
2.3.1	Data Dictionary	14
2.4	Object Diagrams	16
3	Requirements	17
3.1	Software Design Concepts and Design Principles	17
3.1.1	Modularity	17
3.1.2	Abstraction	19
3.1.3	Encapsulation	22
3.1.4	Refactoring	24
3.1.5	SOLID Design Principles	26
3.1.6	Proposed Software Architecture	35
4	Proposed Design Patterns	36
4a	Facade Pattern — Admin Dashboard	36
4b	Strategy Pattern — Checkout Payment	41
4c	State Pattern — Order Lifecycle	46
4d	Factory Method — Role-Based Dashboard Routing	52
4e	Observer Pattern — Notification System	57
5	Conclusion and Suggestion	63
5.1	Conclusion	63
5.2	Suggestion	63
6	Bibliography / References	64

Abstract Summary

This document presents the software design and architectural specification of TechVault, a web-based Consumer Electronics E-Commerce Platform engineered to digitalize and streamline the retail operations of an independent electronics vendor. The platform enables a single vendor to manage a structured product catalog spanning smartphones, laptops, tablets, audio peripherals, and accessories, while delivering a specification-driven shopping experience that simplifies product discovery, comparison, and secure purchase completion for technology consumers.

The system is organized into two primary operational modules. The Customer Module handles product search and filtering by technical attributes, variant selection, shopping cart management, and order lifecycle tracking. The Vendor/Admin Module manages real-time SKU-level inventory across product variants such as color and storage capacity, warranty registration records, customer enquiry responses, and sales analytics reporting. This modular approach, as emphasized by the Institute of Data (2024), enables better code organization and maintainability by dividing a complex system into smaller, independently deployable components.

The architecture applies all five SOLID design principles to ensure low coupling and high cohesion between system components, and implements five design patterns like Facade, Strategy, State to address specific structural and behavioral challenges across the platform. The backend is developed in Python using a Layered (N-Tier) Architecture supported by an SQLite relational database. The frontend is delivered through HTML and CSS templates, keeping the presentation layer entirely separate from business logic.

1. Introduction

1.1 Abstract

TechVault is a standalone, single-vendor consumer electronics e-commerce platform designed to replace fragmented manual sales processes — such as social media order-taking and spreadsheet-based stock tracking — with a centralized, automated web application. The system provides customers with a specification-first product browsing interface, real-time variant-level cart validation, and a complete order lifecycle from payment to delivery confirmation. For the vendor, it provides a unified admin dashboard to manage inventory, respond to warranty enquiries, and generate sales reports.

The software design emphasizes modularity, abstraction, and encapsulation as its core design concepts, and is structured around a Layered Architecture that cleanly separates the Presentation, Business Logic, and Data Access layers. The system incorporates asynchronous order processing and a relational database with strictly enforced referential integrity to ensure data consistency across the multi-variant electronics inventory. As noted by InterSystems (n.d.), relational databases leverage transaction management to guarantee data integrity and synchronization throughout any update or process a critical requirement when managing stock across hundreds of variant SKUs where a single oversell event can result in significant financial loss and customer trust damage.

1.2 Problem Statement

Independent electronics retailers and small-scale gadget resellers currently manage their business operations through fragmented, manual processes. This creates several critical, interconnected problems that limit their operational efficiency and damage the customer experience.

Firstly, Inventory Inconsistency and High Financial Risk. Due to the high unit cost of electronic devices such as laptops, smartphones, and graphics cards, vendors generally hold smaller quantities of stock compared to fast-moving consumer goods. Without real-time stock synchronization at the variant level, vendors frequently experience overselling — a customer pays for a specific device configuration that has already sold out. Processing manual refunds for high-value transactions incurs banking penalties and severely damages consumer trust. This synchronization gap is reflected in the findings of Mahmud Basuki (2021), who states that a lack of connectivity and integration prevents businesses from maintaining a consistent supply chain network, resulting in significant downstream operational disruption.

Next, Fragmented Data and Technical Specification Mismanagement. Electronic devices possess highly complex and varied attributes such as storage capacities, processor generations, and display resolutions. Without a centralized platform for storing structured product data, the vendor cannot present comparable specification sheets to customers, generate accurate sales reports by hardware configuration, or identify top-selling variants. This

issue is supported by Wang (2022), whose research demonstrates that the compartmentalization of data collections creates significant barriers to data sharing, making comprehensive business analysis effectively illusory.

Lastly, Inefficient After-Sales Support and Warranty Tracking. Unlike standard retail goods, electronics require reliable post-purchase support for technical inquiries, defective item returns, and warranty claims. Managing these through manual communication channels such as social media messages or phone calls makes it impossible to maintain a consistent, traceable support record. According to Meyer and Schwager (2007), limiting customer support to ad-hoc, manual post-purchase handling prevents businesses from capturing the subjective aspects of customer feedback that define a high-quality experience, leading to immediate customer churn in a competitive market.

1.3 Project Objectives

The primary objective of the TechVault project is to engineer a secure, scalable single-vendor electronics e-commerce platform that automates complex hardware retail operations, mitigates financial risk through real-time inventory management, and delivers a premium, specification-driven shopping experience for technology consumers. The specific objectives are:

Implement Granular Variant-Level Inventory and Order Management

Unlike standard retail goods, electronic devices have highly detailed technical specifications across variants (e.g., processor generations, RAM capacities, storage configurations). A key objective is to develop a robust inventory management module that tracks stock levels in real time at the SKU variant level, preventing accidental overselling of high-value items. The system will also manage a transparent order lifecycle where customers can monitor their order from 'Paid' through 'Shipped' to 'Received' in real time. As supported by Yousuf (2024), real-time order tracking is a statistically significant driver of customer satisfaction and repeat purchase behavior.

Facilitate Advanced Product Discovery and Comparison

Technology consumers make purchasing decisions based on specific technical requirements rather than visual aesthetics alone. This project aims to design a specification-first catalog interface with keyword search and category-based filtering. Additionally, a Voucher Management System will allow the vendor to issue promotional codes during seasonal campaigns. According to Stan (2024), 65% of consumers are more likely to purchase a product when a discount is offered, rising to 75% among shoppers aged 18 to 30, validating the commercial impact of this feature.

Ensure Secure, Accurate Processing for High-Value Transactions

Because electronics are high-ticket items, financial accuracy and transaction security are paramount. This project aims to construct a reliable checkout pipeline that handles cart validation at the variant level, calculates precise totals including applicable discounts, accepts simulated payment methods such as Credit Card and DuitNow QR, and generates a receipt

upon successful payment. As noted by Lee (2025), technical data accuracy in transaction systems is directly linked to long-term consumer trust in e-commerce platforms.

Provide Actionable Business Analytics for the Vendor

This project aims to enable the vendor to generate and export comprehensive sales reports covering revenue by product category, top-selling variant configurations, and restocking recommendations. As noted by Moreira (2023), technological integration optimizes organizational management by reducing task execution time, freeing business owners for more strategic responsibilities. A centralized analytics dashboard eliminates the error-prone process of compiling sales data from fragmented sources.

1.4 Literature Review

Kumar Ravi Ranjan investigated the integration of electronic commerce into business operations across finance, marketing, and information systems, arguing that e-commerce functions not merely as a sales channel but as a fundamental catalyst for organisational transformation [1]. The study classified e-commerce activity into four core models B2B, B2C, C2C, and C2B and found that the B2B segment was experiencing the most rapid growth, attributed to its capacity to automate corporate procurement and supply chain processes at scale [1]. A particularly significant finding was that the digitalisation of products and services could reduce order processing times by over 90%, resulting in substantial reductions in capital costs [1]. The author further noted that this compression of operational overhead enables small businesses to enter and compete in new markets without the financial burden of physical infrastructure investment [1]. These findings are directly relevant to the objectives of TechVault. The efficiency gains identified in [1] support the case for automating the complete order lifecycle from product listing and stock management through to order fulfilment rather than relying on manual, error-prone processes. The literature suggests that such automation is not simply a technical convenience but a strategic prerequisite for smaller retailers seeking to remain competitive in a marketplace increasingly defined by speed and operational efficiency.

Lim et al. investigated the determinants of online purchase intention among consumers in the consumer electronics category [2]. Drawing on survey data, the study found that product information quality operationalised as the availability of detailed technical specifications and the capacity for structured product comparison was the single strongest predictor of purchase intent within this segment [2]. The authors distinguished between high-involvement product categories, such as laptops and smartphones, and lower-involvement commodity purchases, finding that consumers in the former category required significantly more structured information before committing to a transaction [2]. Platforms that failed to provide this level of detail were found to experience materially higher cart abandonment rates [2]. The study also identified transparency around return policies, warranty terms, and secure payment indicators as critical trust signals in the conversion of browsing behaviour into completed purchases [2]. The implications of these findings for e-commerce platform design are substantial. Lim et al. suggest that for a retailer operating in the consumer electronics space, a specification-first approach to product presentation is not a design preference but an evidence-based requirement [2]. Similarly, their emphasis on post-purchase assurance factors such as warranty clarity and return policy visibility provides empirical justification for the integration of warranty tracking and after-sales enquiry management as core platform features rather than peripheral additions.

Castañó-Herrera et al. investigated the ways in which digital e-commerce platforms are restructuring the distribution landscape for consumer electronics, with a focus on shifting consumer mobility behaviour and evolving supply chain dynamics [3]. Employing a Generalised Structural Equation Model, the study found that the increasing availability of detailed product information and competitive pricing on digital platforms had significantly reduced the frequency of in-store visits for electronics purchases, with urban consumer segments showing a marked tendency to complete their entire purchase journey online [3]. Among post-purchase variables, delivery speed and return policy transparency emerged as the two most influential factors in determining repeat purchase behaviour and long-term customer retention in the electronics segment [3]. The authors concluded that competitive viability in this market requires sustained investment in both pre-purchase information architecture and post-purchase logistics and support infrastructure [3]. Castañó-Herrera et al.'s findings validate both the market context and the specific design priorities of TechVault [3]. The documented shift toward fully digital purchase journeys confirms the commercial relevance of a dedicated e-commerce platform for electronics retail. More specifically, their identification of delivery transparency and post-purchase support as the primary drivers of customer loyalty provides direct justification for TechVault's inclusion of real-time order tracking and an integrated enquiry and warranty management system as functional requirements central to the platform's design.

1.5 Project and System Scope

TechVault Consumer Electronics E-Commerce System													
Phase	Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Requirements & Planning	Problem Statement												
	Project Objectives												
	Literature Review												
Design Phase	System Overview												
	Use Case Diagram												
	Class Diagram												
	ERD Design												
	Object Diagram												
Design Patterns	Design Principles (SOLID)												
	Pattern 1 — Façade												
	Pattern 2 — Strategy Pattern												
	Pattern 3 — State Pattern												
Development & Testing	Frontend Development												
	Backend Development												
	Testing & Debugging												
Final Documentation	Conclusion & References												
	PDF Formatting & Submission												

This Gantt Chart shows the overall schedule for the TechVault Consumer Electronics E-Commerce System and provides a clear presentation of the tasks that are planned for each week. This overall project is divided into 5 phases, which are Requirements & Planning, Design Phase, Design Patterns, Development and Testing and Final Documentation. During this project aside from the Final Documentation Phase, there are task in other phases that are documented, such as the problem statement, project objectives, literature review, system overview, use case diagrams, ERD design diagrams, Object diagrams, Design Principles and the three design patterns. During development, we will using HTML nad CSS for the frontend development, while using Python during the backend development. The tasks in this chart is scheduled to ensure that each task is given enough time to be completed.

In Scope

The TechVault Consumer Electronics E-Commerce System is a standalone, single-vendor web application designed to automate the full retail lifecycle of electronic devices, from specification-driven product discovery through to order fulfillment and warranty activation. The following functional modules are within scope:

- **User and Profile Management:** Secure authentication (Sign-up / Sign-in), persistent storage of user profiles including home addresses and contact details, and a personal dashboard showing purchase history and active warranties.
- **Product Catalog with Specification Display:** Category-based browsing (Smartphones, Laptops, Audio, Accessories) with keyword search and filtering by technical attributes. Each product supports multiple variants with distinct attributes such as color and storage capacity.
- **Variant-Level Inventory and Shopping Cart:** Real-time stock synchronization at the SKU variant level. Strict validation prevents users from adding more units than available stock. Cart items are stored persistently until checkout.
- **Transaction Processing (Checkout):** Cart total calculation, voucher code application, simulated payment methods (Credit Card, FPX, DuitNow QR), and receipt generation upon successful payment.
- **Order Lifecycle Tracking:** A state-managed order system with statuses — Paid, Shipped, and Received — visible to both the customer and the vendor in real time.
- **Warranty Registration and Enquiry System:** Customers can register purchased products for warranty and submit technical support enquiries. The vendor responds through the admin dashboard.
- **Admin Dashboard:** Centralized vendor interface for managing product listings, updating variant stock, processing orders, responding to enquiries, and generating sales reports.

Out of Scope

To ensure the project remains focused and feasible within the given development timeframe, the following have been explicitly excluded:

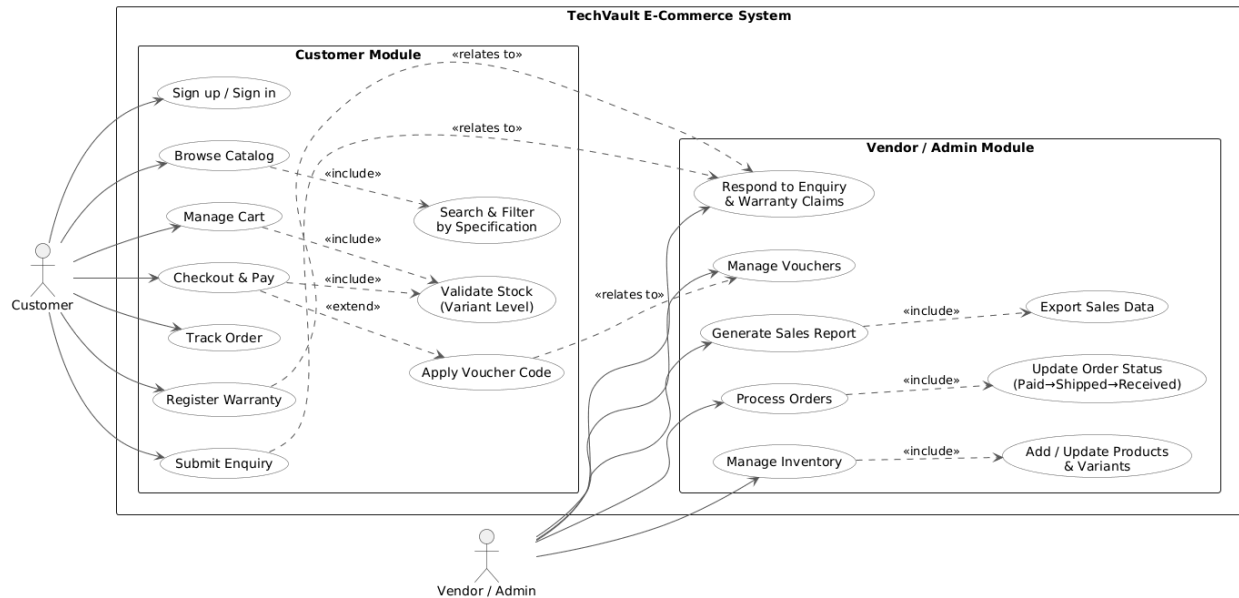
- **Live Payment Gateway Integration:** The system uses a mock payment interface. Real banking integrations such as Stripe, PayPal, or FPX Direct require complex PCI-DSS compliance that is out of scope for this iteration.
- **Third-Party Logistics with Live Tracking:** Order statuses are managed internally by the admin. The system does not integrate with external delivery APIs such as DHL, FedEx, or NinjaVan for real-time GPS parcel tracking.
- **Multi-Vendor Marketplace:** The platform is strictly a single-vendor system. It does not support a multi-seller marketplace structure like Amazon or Shopee where independent sellers can register and list their own products.
- **External Automated Notifications:** All system notifications such as order confirmation and shipping updates are displayed only within the application interface. No integration with SMTP email servers or SMS gateways is included.

- POS and Warehouse Integrations: The platform functions purely as a digital storefront with no connections to barcode scanners, POS terminals, or physical warehouse automation systems.

2. System Overview

2.1 Generic Use Case

The table and use diagram below outlines the primary actors and their key interactions with the TechVault system. It provides a high-level summary of the intended functional behavior from each actor's perspective.

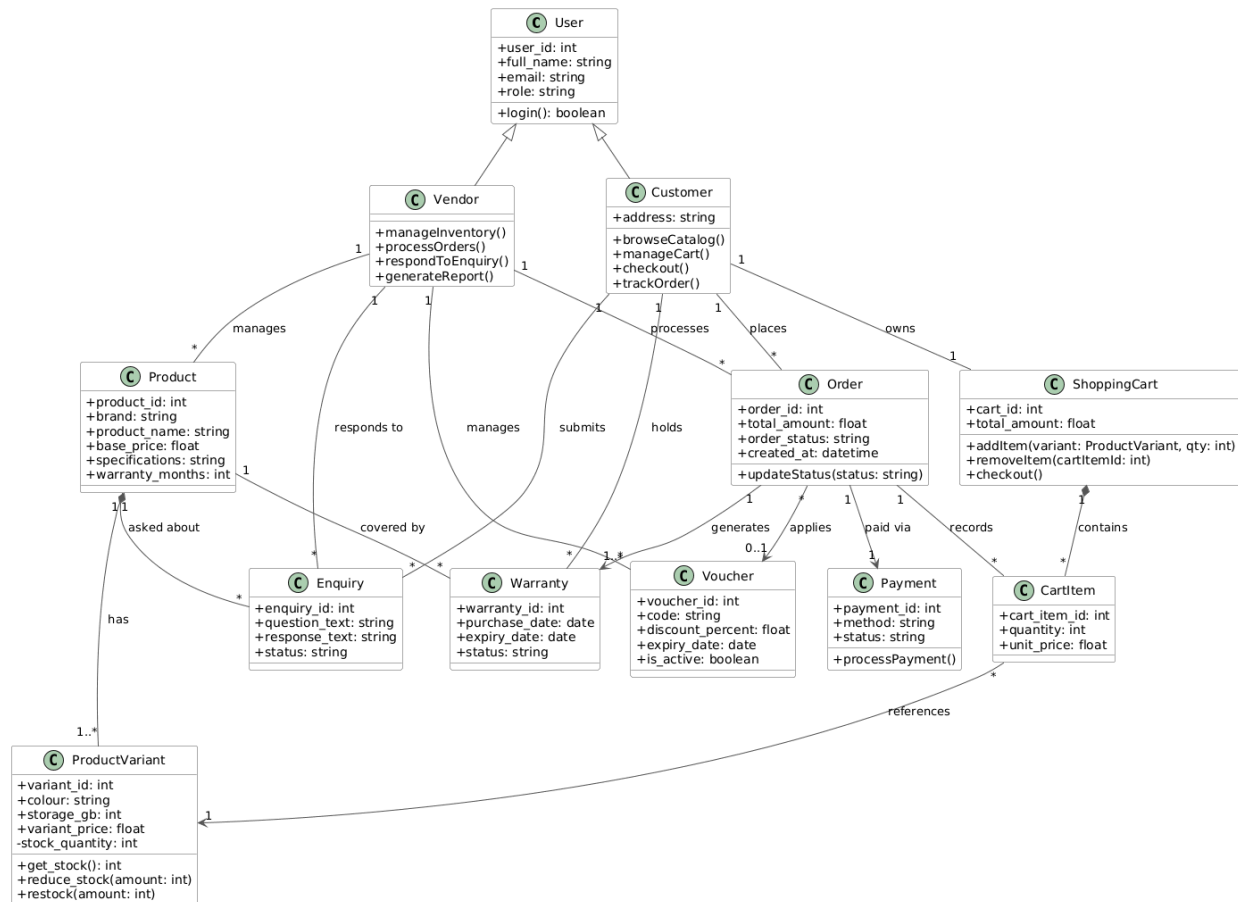


Actor	Use Case	Description	Pre-Condition
Customer	Browse Catalog	View products, search by keyword, filter by category or specification.	User arrives at the platform.
Customer	Manage Cart	Add or remove product variants from the shopping cart.	Selected variant must be in stock.
Customer	Checkout & Pay	Enter shipping info, apply voucher, choose payment method, complete order.	Logged in. Cart items > 0.
Customer	Track Order	View real-time status of current and past orders.	Must be logged in.
Customer	Register Warranty	Submit warranty registration for a completed purchase.	Logged in. Order status: Received.
Customer	Submit Enquiry	Submit a technical support question about a product.	Must be logged in.

Vendor	Manage Inventory	Add products, manage variants, update stock, remove discontinued items.	Logged in as Administrator.
Vendor	Process Orders	Update order statuses from Paid to Shipped.	Logged in as Administrator.
Vendor	Respond to Enquiry	View and respond to customer warranty claims and technical enquiries.	Logged in as Administrator.
Vendor	Generate Report	Export sales reports by category, variant, and revenue period.	Logged in as Administrator.

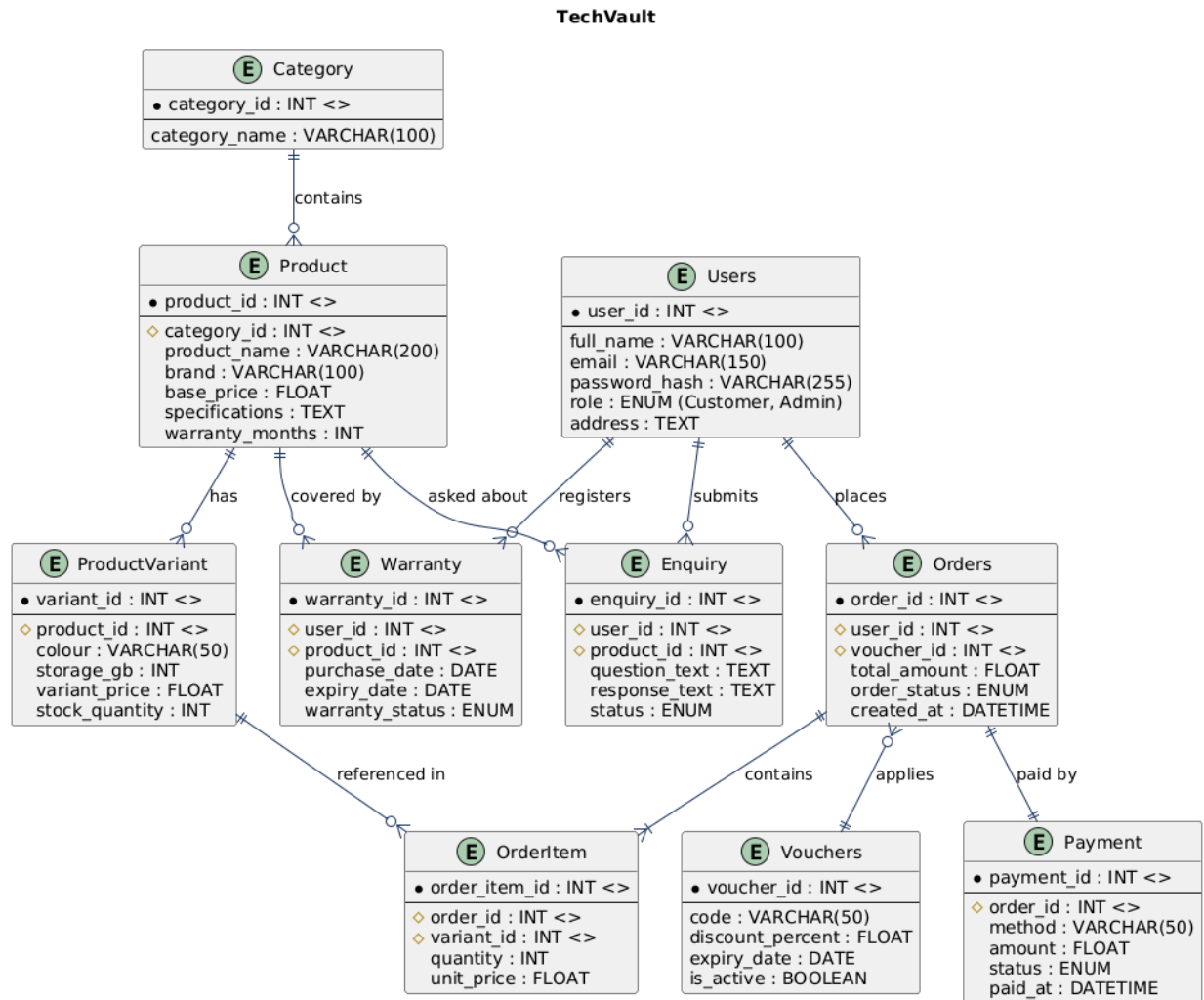
2.2 Class Diagrams

The class diagram below illustrates the static structure of the TechVault system, showing its core classes, attributes, methods, and the structural relationships between them. The diagram reflects the two-module architecture (Customer and Admin) and the variant-level inventory model required for electronics retail with multiple SKUs per product.



2.3 Entity Relationship Diagram (ERD)

The ERD below visualizes the database schema for TechVault. It documents all entities and their relationships, reflecting the multi-variant product model and the full order lifecycle tracking required for a consumer electronics platform.



2.3.1 Data Dictionary

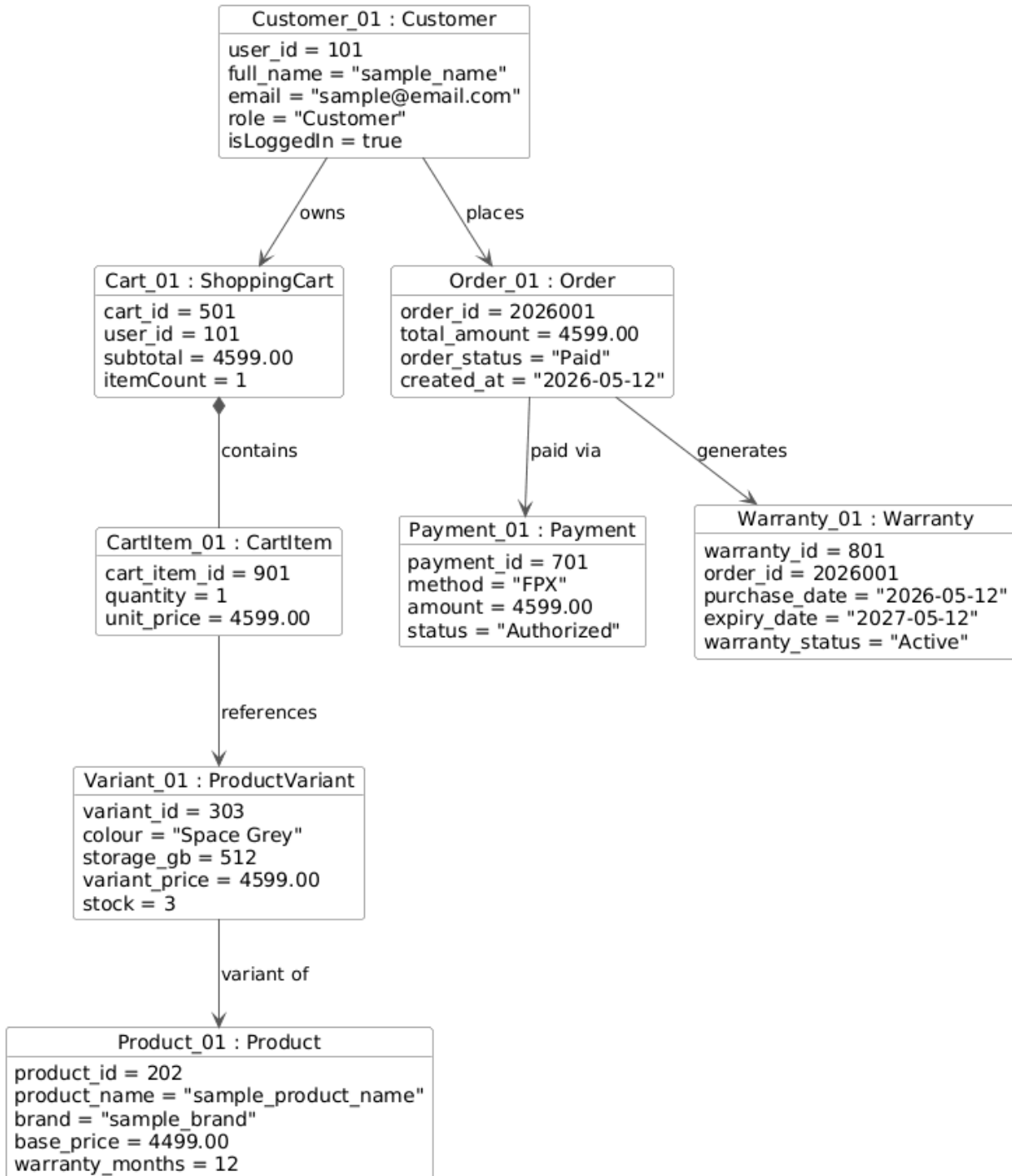
The data dictionary below defines all data attributes used within the system's primary database tables.

Entity	Attribute Name	Data Type	Key	Description / Reference
Users	user_id	INT	PK	Unique user identifier
	full_name	VARCHAR(100)	—	Customer or vendor full name
	email	VARCHAR(150)	—	Unique login email address
	password_hash	VARCHAR(255)	—	Hashed user password
	role	ENUM	—	Customer or Admin
	address	TEXT	—	Default shipping address
Product	product_id	INT	PK	Unique product identifier
	category_id	INT	FK	References Category table
	product_name	VARCHAR(200)	—	Full product model name
	brand	VARCHAR(100)	—	Manufacturer brand name
	base_price	FLOAT	—	Starting retail price
	specifications	TEXT / JSON	—	Technical specification data
	warranty_months	INT	—	Default warranty period in months
ProductVariant	variant_id	INT	PK	Unique SKU identifier
	product_id	INT	FK	References Product table
	colour	VARCHAR(50)	—	Variant colour option
	storage_gb	INT	—	Storage capacity (e.g., 128, 256, 512)
	variant_price	FLOAT	—	Price adjustment for this variant
	stock_quantity	INT	—	Current available stock for this SKU
Category	category_id	INT	PK	Unique category identifier

	category_name	VARCHAR(100)	—	e.g., Smartphone, Laptop, Audio
Orders	order_id	INT	PK	Unique order identifier
	user_id	INT	FK	References Users table
	voucher_id	INT	FK	References Vouchers (nullable)
	total_amount	FLOAT	—	Final order total after discounts
	order_status	ENUM	—	Paid, Shipped, Received, Refunded
	created_at	DATETIME	—	Order creation timestamp
Warranty	warranty_id	INT	PK	Unique warranty record identifier
	user_id	INT	FK	References Users table
	product_id	INT	FK	References Product table
	purchase_date	DATE	—	Date of completed purchase
	expiry_date	DATE	—	Calculated warranty expiry date
	warranty_status	ENUM	—	Active, Expired, or Claimed
Enquiry	enquiry_id	INT	PK	Unique enquiry identifier
	user_id	INT	FK	References Users table
	product_id	INT	FK	References Product table
	question_text	TEXT	—	Customer support question body
	response_text	TEXT	—	Vendor's response (nullable until replied)
	status	ENUM	—	Pending or Resolved

2.4 Object Diagrams

The object diagram below provides a concrete runtime snapshot of the TechVault system during an active checkout scenario. It illustrates a specific instance where a logged-in customer (AlexTan) purchases a laptop variant, completes payment via FPX, the vendor ships the order, and a warranty record is automatically generated upon delivery confirmation.



This snapshot demonstrates that all object instances are correctly linked at runtime, validating that the relational database foreign key constraints and the object-oriented class associations defined in the class diagram behave correctly as a unified system.

3. Requirement

3.1 Software Design Concepts and Design Principles

This section outlines the five key software design concepts applied in the construction of the TechVault system, followed by a detailed application of all five SOLID design principles. Together, these govern every architectural decision made across the codebase.

3.1.1 Modularity

Modularity refers to the division of a system into smaller, logically independent components that each handle a specific responsibility. This approach enables more efficient development planning, easier accommodation of future changes, and targeted testing and debugging without impacting unrelated system components. As stated by the Institute of Data (2024), modularity is essential for maintaining code readability and allowing individual parts of a system to be updated or replaced without disrupting the whole.

In TechVault, modularity is applied by separating the system into four distinct backend modules: the Customer Module (product browsing, cart, checkout, order tracking), the Admin Module (inventory management, order processing, report generation), the Warranty Module (warranty registration and status tracking), and the Notification Module (internal status updates). This structure allows, for example, the Warranty Module to be independently updated to support extended warranty period calculations without any impact on the checkout or notification logic.

Code Example:

```
# customer_module.py
class CustomerModule:
    def browse_products(self):
        return "Displaying product catalog"

    def add_to_cart(self, product_variant_id, quantity):
        return f"Added variant {product_variant_id} x{quantity} to cart"

    def checkout(self):
        return "Proceeding to checkout"

    def track_order(self, order_id):
        return f"Tracking order {order_id}"
```

```

# admin_module.py
class AdminModule:
    def manage_inventory(self):
        return "Managing product inventory"

    def process_orders(self, order_id, new_status):
        return f"Order {order_id} updated to {new_status}"

    def generate_report(self):
        return "Sales report generated"

# warranty_module.py
class WarrantyModule:
    def register_warranty(self, user_id, product_id):
        return f"Warranty registered for user {user_id} and product {product_id}"

    def check_warranty_status(self, warranty_id):
        return f"Warranty {warranty_id} is Active"

# notification_module.py
class NotificationModule:
    def send_internal_update(self, message):
        return f"Notification sent: {message}"

# main_app.py
from customer_module import CustomerModule
from admin_module import AdminModule
from warranty_module import WarrantyModule
from notification_module import NotificationModule

class TechVaultApp:
    def __init__(self):
        self.customer_module = CustomerModule()
        self.admin_module = AdminModule()
        self.warranty_module = WarrantyModule()
        self.notification_module = NotificationModule()

```

```
def demo(self):
    print(self.customer_module.browse_products())
    print(self.customer_module.add_to_cart(101, 2))
    print(self.admin_module.manage_inventory())
    print(self.warranty_module.register_warranty(1, 101))
    print(self.notification_module.send_internal_update("Order has been shipped"))
```

The modular design is implemented by dividing the TechVault system into multiple independent service classes, where each module handles a specific business domain. The CustomerModule manages user-facing operations such as browsing products, adding items to the cart, and tracking orders. The AdminModule handles backend operations including inventory management, order processing, and report generation. Meanwhile, the WarrantyModule is responsible for warranty registration and tracking, and the NotificationModule manages internal system notifications.

These modules are instantiated and coordinated through a central TechVaultApp class, which acts as a composition root. Each module operates independently and communicates only when required, ensuring low coupling between system components

3.1.2 Abstraction

Abstraction hides the unnecessary internal complexity of a system and exposes only the essential interface needed by users or other components. It reduces cognitive load on developers and consumers of a module by allowing them to interact with simplified, high-level interfaces rather than low-level implementation details.

In TechVault, abstraction is applied extensively. When a customer adds a product variant to their cart, they simply click 'Add to Cart'. The system internally executes stock validation against the variant's SKU, resolves the applicable pricing, and recalculates the cart total — all hidden behind a clean, abstract CartService interface. Similarly, the vendor interacts with a simple InventoryManager interface to update stock, without needing to directly execute database queries or understand SKU resolution logic.

Code Example:

```
# abstraction.py

from abc import ABC, abstractmethod


class CartService(ABC):
```

```

    @abstractmethod
    def add_item(self, product_variant_id: int, quantity: int):
        pass

    @abstractmethod
    def calculate_total(self):
        pass

class InventoryManager(ABC):
    @abstractmethod
    def check_stock(self, product_variant_id: int, quantity: int) -> bool:
        pass

    @abstractmethod
    def get_variant_price(self, product_variant_id: int) -> float:
        pass

class InventoryManager(MyInventoryManager):
    def __init__(self):
        self.stock = {
            1: {"price": 4599.00, "qty": 10},
            2: {"price": 2899.00, "qty": 5},
        }

    def check_stock(self, product_variant_id: int, quantity: int) -> bool:
        return self.stock.get(product_variant_id, {}).get("qty", 0) >= quantity

    def get_variant_price(self, product_variant_id: int) -> float:
        return self.stock.get(product_variant_id, {}).get("price", 0.0)

class CartService(MyCartService):

```

```

def __init__(self, inventory: IInventoryManager):
    self.inventory = inventory
    self.items = []

def add_item(self, product_variant_id: int, quantity: int):
    if not self.inventory.check_stock(product_variant_id, quantity):
        raise ValueError("Insufficient stock for this variant")

    price = self.inventory.get_variant_price(product_variant_id)
    self.items.append({
        "variant_id": product_variant_id,
        "quantity": quantity,
        "unit_price": price
    })

    return f"Variant {product_variant_id} added to cart"

def calculate_total(self):
    total = sum(item["unit_price"] * item["quantity"] for item in self.items)
    return round(total, 2)

# usage
inventory = InventoryManager()
cart = CartService(inventory)

print(cart.add_item(1, 1))
print("Cart total:", cart.calculate_total())

```

The code implements a simple shopping cart system using the concept of abstraction. Two abstract classes, `MyCartService` and `MyInventoryManager`, are defined to specify the required methods such as adding items, checking stock availability, retrieving product prices, and calculating the total cost. These classes only outline the required functionalities without providing their implementation. The `InventoryManager` class implements `MyInventoryManager` by maintaining product details, including price and quantity, and provides methods to verify stock availability and retrieve pricing information. The `CartService`

class implements `MyCartService` and interacts with the inventory to ensure that sufficient stock is available before adding items to the cart. It stores selected items and calculates the total cost by summing the product of item price and quantity. This separation between definition and implementation improves code organization, maintainability, and flexibility.

3.1.3 Encapsulation

Encapsulation bundles data and the methods that operate on it within a single class while restricting direct external access to the internal state. This prevents unauthorized or accidental modification of critical data and ensures that all state changes go through controlled, validated pathways.

In TechVault, encapsulation is critical for protecting sensitive attributes such as variant-level stock quantities, order lifecycle states, and warranty validity periods. The `ProductVariant` class below demonstrates this principle: the stock quantity is declared as a private attribute, accessible only through validated public methods that enforce business rules before making any modification.

Code Example:

```
# product_variant.py

class ProductVariant:
    def __init__(self, variant_id: int, colour: str, storage_gb: int, stock: int,
price: float):
        self.variant_id = variant_id
        self.colour = colour
        self.storage_gb = storage_gb
        self.__stock = stock          # private attribute
        self.__price = price          # private attribute

    # Getter for stock
    def get_stock(self) -> int:
        return self.__stock

    # Getter for price
    def get_price(self) -> float:
        return self.__price
```

```

# Controlled method to reduce stock
def reduce_stock(self, amount: int):
    if amount <= 0:
        raise ValueError("Reduction amount must be positive")

    if amount > self.__stock:
        raise ValueError("Insufficient stock for this variant")

    self.__stock -= amount
    print(f"Stock reduced by {amount}. Remaining stock: {self.__stock}")

# Controlled method to increase stock
def restock(self, amount: int):
    if amount <= 0:
        raise ValueError("Restock amount must be positive")

    self.__stock += amount
    print(f"Stock increased by {amount}. Current stock: {self.__stock}")

# Controlled method to update price
def update_price(self, new_price: float):
    if new_price <= 0:
        raise ValueError("Price must be greater than zero")

    self.__price = new_price
    print(f"Price updated to RM{self.__price:.2f}")

```

The code defines a `ProductVariant` class that represents a specific version of a product with attributes such as variant ID, colour, storage size, stock, and price. The stock and price are declared as private attributes to ensure they cannot be accessed or modified directly from outside the class. Instead, getter methods are provided to safely retrieve their values. The class also includes controlled methods such as `reduce_stock`, `restock`, and `update_price`, which manage changes to stock and price while enforcing validation rules. For example, it prevents reducing stock below zero, disallows negative or zero values for restocking, and ensures that the price remains positive. This approach demonstrates encapsulation by

protecting the internal data and allowing modifications only through controlled and validated methods, thereby improving data integrity and system reliability.

3.1.4 Refactoring

Refactoring is the process of restructuring existing code to improve its internal clarity, reduce duplication, and improve maintainability without changing its external behavior. In an electronics e-commerce system, refactoring is especially important during iterative development of complex features such as variant pricing calculations and warranty expiry logic, where initial implementations tend to produce tightly coupled, duplicated code.

In TechVault, refactoring is applied by extracting repeated business logic — such as variant discount calculations that previously appeared in both the cart total method and the checkout service — into reusable utility functions. This ensures a single source of truth for pricing logic and reduces the risk of calculation inconsistencies when business rules change.

Code Example:

```
# pricing_utils.py

def calculate_discounted_price(base_price: float, variant_price: float = 0.0,
discount_percent: float = 0.0) -> float:
    """
    Shared pricing function used by both cart and checkout.
    This avoids duplicated logic in multiple services.
    """
    subtotal = base_price + variant_price
    discount_amount = subtotal * (discount_percent / 100)
    final_price = subtotal - discount_amount
    return round(final_price, 2)

# cart_service.py
from pricing_utils import calculate_discounted_price

class CartService:
    def __init__(self):
        self.items = []

    def add_item(self, product_name: str, base_price: float, variant_price: float,
discount_percent: float):
        final_price = calculate_discounted_price(base_price, variant_price,
discount_percent)
```

```

        self.items.append({
            "product_name": product_name,
            "final_price": final_price
        })

        return f"{product_name} added to cart at RM{final_price:.2f}"

    def get_cart_total(self):
        return round(sum(item["final_price"] for item in self.items), 2)

# checkout_service.py
from pricing_utils import calculate_discounted_price

class CheckoutService:
    def __init__(self):
        self.receipt_items = []

    def add_checkout_item(self, product_name: str, base_price: float, variant_price:
float, discount_percent: float):
        final_price = calculate_discounted_price(base_price, variant_price,
discount_percent)

        self.receipt_items.append({
            "product_name": product_name,
            "final_price": final_price
        })

    def calculate_grand_total(self):
        return round(sum(item["final_price"] for item in self.receipt_items), 2)

    def generate_receipt(self):
        total = self.calculate_grand_total()

        return f"Receipt generated. Total amount: RM{total:.2f}"

# demo_refactoring.py

```

```

from cart_service import CartService
from checkout_service import CheckoutService

cart = CartService()
checkout = CheckoutService()

print(cart.add_item("Laptop M3", 4299.00, 300.00, 10))
checkout.add_checkout_item("Laptop M3", 4299.00, 300.00, 10)

print("Cart Total:", cart.get_cart_total())
print(checkout.generate_receipt())

```

The code demonstrates the use of a shared function to avoid repeating the same logic in multiple parts of the system. The `calculate_discounted_price` function is created in a separate file to handle price calculation by combining the base price and variant price, applying a discount, and returning the final value. This function is then reused in both `CartService` and `CheckoutService`, ensuring that the pricing calculation remains consistent throughout the system. The `CartService` uses it when adding items to the cart and calculating the total cost, while the `CheckoutService` uses it to process checkout items, calculate the grand total, and generate a receipt. This approach improves code organization, reduces duplication, and makes the system easier to maintain.

3.1.5 SOLID Design Principles

The following five SOLID principles are applied throughout the TechVault codebase to ensure each component is focused, extensible, and easy to maintain.

Single Responsibility Principle (SRP)

A class should have only one reason to change. In TechVault, this is applied by separating order saving, warranty registration, and notification dispatch into dedicated classes. An `OrderRepository` handles only database persistence; a `WarrantyService` handles only warranty logic, a `NotificationService` handles only alert delivery. No single class is responsible for more than one of these concerns.

The code demonstrates the Single Responsibility Principle (SRP) by showing a comparison before and after applying it. In the initial version, the `OrderManager` class handles multiple responsibilities in one place, including saving the order, registering warranty, and sending customer notifications, which makes the code less organized and harder to maintain. After applying SRP, these responsibilities are separated into different classes: `OrderRepository` handles saving orders to the database, `WarrantyService` manages warranty registration, and `NotificationService` is responsible for sending notifications to customers. The

OrderProcessor class then coordinates these services to process an order by calling each class accordingly. This separation of concerns makes the code cleaner, easier to maintain, and more flexible for future changes.

Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. In TechVault, this is applied to the payment module. New payment methods can be added by creating a new concrete class implementing the MyPaymentMethod interface, without ever modifying the existing PaymentService or other strategy classes. This directly supports TechVault's requirement to add Malaysian-specific payment options such as FPX and DuitNow without disrupting existing card payment logic.

```
#Before applying OCP

class PaymentService:

    def pay(self, method, amount):

        if method == "card":

            print("Paying with card")

        elif method == "fpx":

            print("Paying with FPX")

        elif method == "duitnow":

            print("Paying with DuitNow")


#After applying OCP

from abc import ABC, abstractmethod

class PaymentStrategy(ABC):

    @abstractmethod

    def pay(self, amount):

        pass


class CardPayment(PaymentStrategy):

    def pay(self, amount):

        print(f"Paid RM{amount} using Card")
```

```

class FPXPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid RM{amount} using FPX")

class DuitNowPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid RM{amount} using DuitNow")

class PaymentService:
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def process_payment(self, amount):
        self.strategy.pay(amount)

```

The code demonstrates the Open/Closed Principle (OCP) by showing how the payment system is improved before and after refactoring. In the initial version, the `PaymentService` class handles different payment methods using multiple `if-else` statements, which makes the code harder to maintain because any new payment method requires modifying the existing code. After applying OCP, a `PaymentStrategy` abstract class is introduced to define a common interface for all payment methods. Each payment type, such as `CardPayment`, `FPXPayment`, and `DuitNowPayment`, is implemented in its own class. The `PaymentService` class is then updated to use a selected payment strategy without changing its internal logic. This makes the system easier to extend, as new payment methods can be added without modifying existing code, improving flexibility and maintainability.

Liskov Substitution Principle (LSP)

Objects of a derived class must be substitutable for their parent class without altering the program's correctness. In TechVault, all checkout option types `CreditCardCheckout`, `FPXCheckout`, and `CashOnDeliveryCheckout` extend a base `Checkout` option class and can be used interchangeably by the `CheckoutService`. A violation would occur if one subclass threw an unexpected exception when the service called its common `complete_checkout()` method.

```

#Before applying LSP

class Checkout:
    def process(self):

```

```

        print("Processing checkout")

class CashOnDelivery(Checkout):
    def process(self):
        raise Exception("COD not supported here") # breaks expectation

#After applying LSP
class Checkout:
    def process(self):
        pass

class CreditCardCheckout(Checkout):
    def process(self):
        print("Processing credit card payment")

class FPXCheckout(Checkout):
    def process(self):
        print("Processing FPX payment")

class CashOnDelivery(Checkout):
    def process(self):
        print("Processing COD order")

```

The code demonstrates the Liskov Substitution Principle (LSP) by showing a problem before applying it and a proper solution after. In the initial version, the `CashOnDelivery` class inherits from `Checkout` but breaks the expected behavior by raising an exception, which means it cannot properly replace the parent class without causing errors. This violates LSP because subclasses should behave in a way that does not change the expected functionality of the parent class. After applying LSP, the `Checkout` class is used as a base structure, and each payment type such as `CreditCardCheckout`, `FPXCheckout`, and `CashOnDelivery` properly implements the `process` method according to its own behavior. This ensures that any subclass can replace the parent class without causing unexpected issues, making the system more reliable and consistent.

Interface Segregation Principle (ISP)

No class should be forced to implement methods it does not use. In TechVault, instead of one large combined interface, responsibilities are split into focused interfaces: `MyPaymentProcessor` (`process_payment`, `refund_payment`), `MyWarrantyManager` (`register_warranty`, `check_status`), and `MyReportGenerator` (`generate_sales_report`). A payment service class implements only `MyPaymentProcessor` and is never forced to implement warranty or reporting methods.

```
# Before applying ISP
class ECommerceService:
    def process_payment(self): pass
    def register_warranty(self): pass
    def generate_report(self): pass

class PaymentService(ECommerceService):
    def process_payment(self):
        print("Processing payment")

    def register_warranty(self):
        pass

    def generate_report(self):
        pass

# After applying ISP
from abc import ABC, abstractmethod

class MyPaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount):
```

```

        pass

class MyWarrantyManager(ABC):
    @abstractmethod
    def register_warranty(self, product_id, user_id):
        pass

class MyReportGenerator(ABC):
    @abstractmethod
    def generate_report(self):
        pass

class PaymentService(MyPaymentProcessor):
    def process_payment(self, amount):
        print(f"Processing payment of RM{amount}")

```

The code demonstrates the Interface Segregation Principle (ISP) by comparing a poor design before refactoring and an improved structure after applying the principle. In the initial version, the `ECommerceService` interface forces implementing classes to include methods that are not always needed, such as `process_payment`, `register_warranty`, and `generate_report`, which leads to unused or empty methods in the `PaymentService` class. After applying ISP, the large interface is split into smaller, more specific interfaces: `MyPaymentProcessor`, `MyWarrantyManager`, and `MyReportGenerator`, each focusing on a single responsibility. The `PaymentService` class now only implements the `MyPaymentProcessor` interface, which matches its actual functionality. This separation ensures that classes only depend on methods they truly need, making the system cleaner, more flexible, and easier to maintain.

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. In TechVault, the high-level CheckoutController does not directly instantiate a specific notification service. Instead, it depends on the abstract NotificationService interface, allowing the concrete delivery mechanism email, SMS, or in-app to be injected at runtime without modifying the checkout logic.

```
#Before applying DIP

class EmailNotification:
    def send(self, message):
        print(f"Email: {message}")

class CheckoutService:
    def __init__(self):
        self.notifier = EmailNotification() # tightly coupled

    def complete_order(self):
        self.notifier.send("Order placed")

#After applying DIP

from abc import ABC, abstractmethod

class NotificationService(ABC):
    @abstractmethod
    def send(self, message):
        pass
```

```

class EmailNotification(NotificationService):
    def send(self, message):
        print(f"Email: {message}")

class SMSNotification(NotificationService):
    def send(self, message):
        print(f"SMS: {message}")

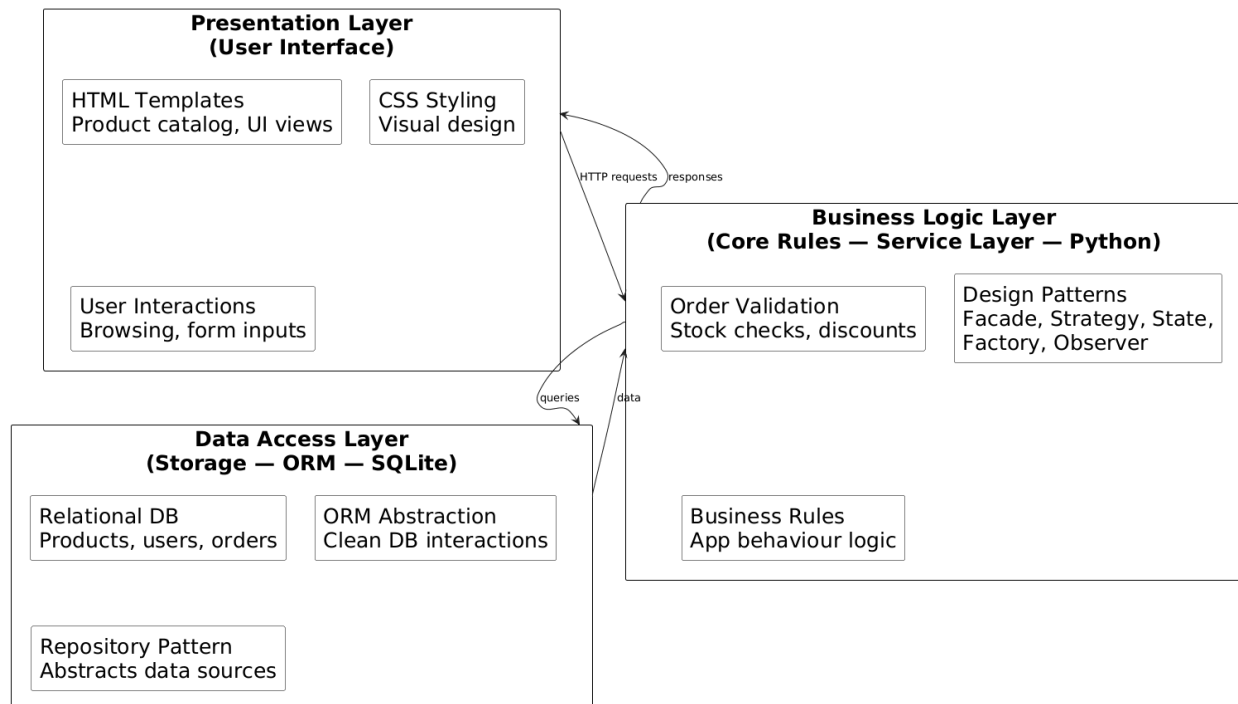
class CheckoutService:
    def __init__(self, notifier: NotificationService):
        self.notifier = notifier

    def complete_order(self):
        self.notifier.send("Order placed successfully")

```

The code demonstrates the Dependency Inversion Principle (DIP) by showing a comparison before and after applying it. In the initial version, the CheckoutService class is tightly coupled to the EmailNotification class, meaning it directly depends on a specific implementation, which makes it difficult to change or extend the notification method. After applying DIP, a NotificationService abstract class is introduced to define a common interface for sending notifications. Both EmailNotification and SMSNotification implement this interface, allowing different notification methods to be used. The CheckoutService is then updated to depend on the abstraction (NotificationService) instead of a concrete class, and the specific notification type is passed into it. This makes the system more flexible, easier to extend, and reduces dependency on specific implementations.

3.1.6 Proposed Software Architecture



The diagram above illustrates how the three layers interact: the Presentation Layer sends HTTP requests down to the Business Logic Layer, which applies all rules and design patterns before issuing queries to the Data Access Layer. Responses flow back up through the same stack, ensuring no layer bypasses its immediate neighbor.

Presentation Layer (Frontend): HTML templates and CSS stylesheets that render the user interface and capture customer inputs such as product browsing, variant selection, and cart actions. This layer communicates only with the Business Logic Layer and never directly accesses the database, ensuring complete decoupling of the UI from application rules.

Business Logic Layer (Backend Services): Python service classes implementing the system's core rules — order processing, warranty registration, stock validation, payment handling, and notification dispatch. This is the central source of truth for all application behavior and mediates all communication between the Presentation Layer and the Data Access Layer.

Data Access Layer (ORM + SQLite): Manages all communication with the SQLite database through an Object Relational Mapper (ORM). By abstracting raw SQL queries behind clean Python methods, this layer reduces the risk of data integrity errors during multi-variant inventory operations and keeps database logic isolated from business rules (Ekpobimi et al., 2024).

4. Proposed Design Patterns

The following five design patterns are proposed to address specific structural and behavioral challenges within the TechVault Consumer Electronics E-Commerce System. Each pattern is selected for its direct relevance to the system's requirements and is presented with a description of its workflow, class and sequence diagrams, sample code, and an analysis of its benefits and limitations.

a. Design Pattern 1: Facade Pattern (Admin Dashboard)

i. Function or Software Component Affected

The Facade Pattern is applied to the Admin Dashboard module which is the centralized vendor management interface of TechVault.

ii. Description of Workflow / Data Flow

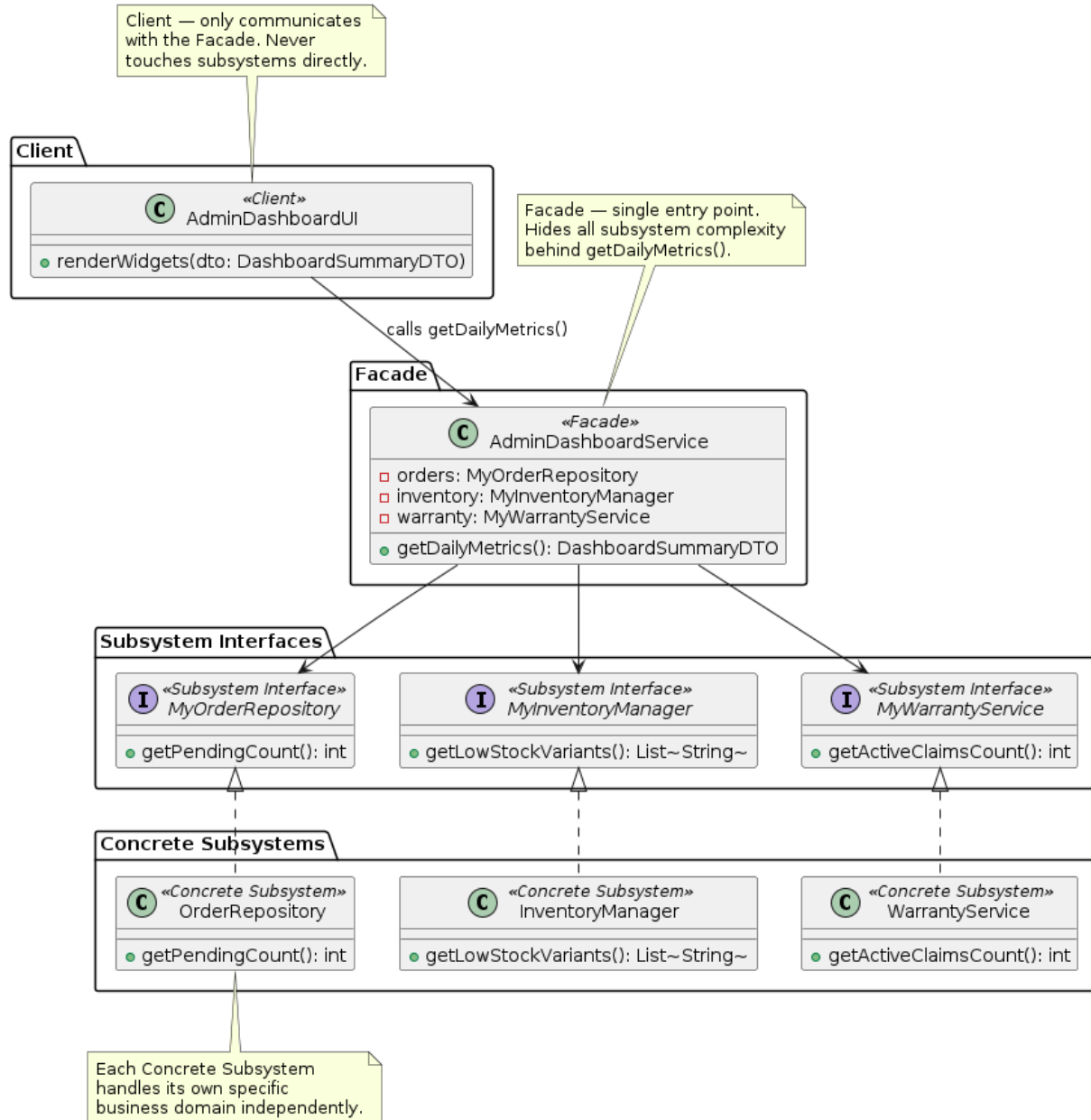
When the Administrator accesses the vendor backend dashboard, the UI requires a comprehensive view of the system's current operational state. Without the Facade Pattern, the frontend would need to make separate, independent calls to the OrderRepository (for pending order counts), the InventoryManager (for low-stock variant alerts), and the WarrantyService (for active warranty claims). This would tightly couple the UI to multiple backend subsystems, making the dashboard fragile and difficult to maintain. With the Facade Pattern, the frontend makes a single call to the AdminDashboardService — the facade. The facade internally queries all required subsystems, aggregates their responses, and returns a single DashboardSummaryDTO to the client. The admin UI never directly accesses any subsystem; all subsystem complexity is hidden behind this unified interface.

iii. Class Diagram

The class diagram illustrates the Facade Pattern applied to the TechVault Admin Dashboard module. At the top of the diagram, the AdminDashboardUI class represents the vendor-facing interface, which contains a single method `renderWidgets(dto: DashboardSummaryDTO)` that receives and displays the dashboard data. The UI communicates exclusively with the AdminDashboardService, labeled with the «Facade» stereotype, by calling its `getDailyMetrics()` method — which is the single entry point highlighted by the note in the diagram. The AdminDashboardService acts as the facade by internally holding private references to three subsystem interfaces: `IOrderRepository`, `IInventoryManager`, and `IWarrantyService`, through which it coordinates all data retrieval without exposing any of that complexity to the UI. Each of these interfaces is implemented by a corresponding concrete class — `OrderRepository`, `InventoryManager`, and `WarrantyService` respectively — shown through the dashed arrows with hollow triangle heads at the bottom of the diagram, representing realization relationships. The facade queries all three subsystems within its `getDailyMetrics()` method, collects the pending order count, low stock variant alerts, and active warranty claims, and packages them into a `DashboardSummaryDTO` return object that is passed back to the UI for rendering. The overall structure demonstrates that

AdminDashboardUI never directly communicates with any subsystem or their interfaces — all internal complexity is hidden behind the facade, keeping the presentation layer clean, decoupled, and easy to maintain.

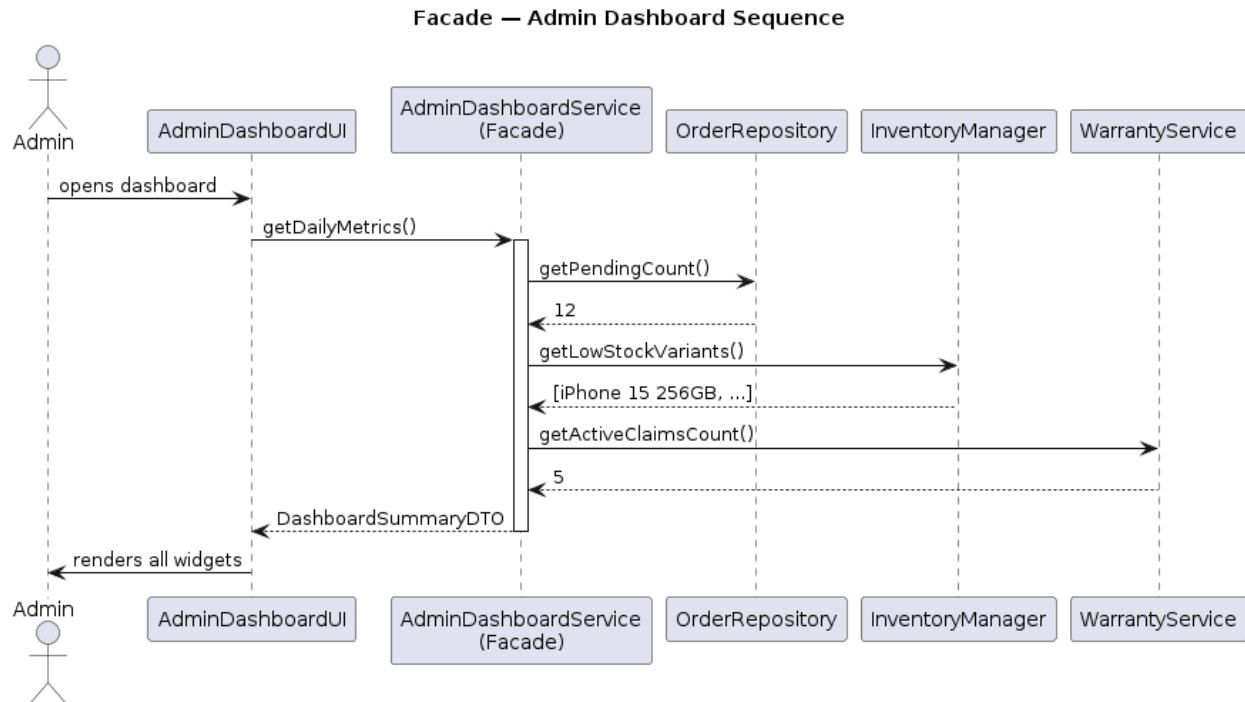
Facade Pattern — Admin Dashboard



iv. Sequence Diagram

This sequence diagram presents how an admin opens the dashboard, and the next part of the sequence diagram shows what happens after that. The dashboard UI makes a single request (`getDailyMetrics()`) to get the data from the admin dashboard service via an admin dashboard service facade. After receiving the request, the admin dashboard service facade makes three separate calls to obtain the required data: it calls `getPendingCount()` against the order repository, it calls `getLowStockVariants()` against the inventory manager, and it calls

getActiveClaimsCount() against the warranty service. Once the admin dashboard service facade receives the results of all three calls, it combines these results into a single summary (DashboardSummaryDTO) and returns this summary to the dashboard UI, which can then render the various dashboard widgets at the same time. The admin does not see how the internal mechanisms of the system are coordinated.



v. Sample Potential Code

iv. Benefits

By making all internal subsystems private and accessible only through the Facade, the admin dashboard UI is completely decoupled from backend complexity. The single public `GetDailyMetrics()` method makes the frontend simpler to develop and maintain. Security is also significantly enhanced, as subsystems cannot be called directly from the presentation layer, preventing unauthorized or unintended data access.

iv. Limitations

If not carefully managed, the Facade class can evolve into a large, bloated component that is tightly coupled to every backend subsystem. As the number of dashboard widgets grows, the facade risks becoming a bottleneck that is difficult to test and maintain independently. This can be mitigated by ensuring the facade delegates cleanly to well-defined subsystem interfaces rather than embedding logic directly.

b. Design Pattern 2: Strategy Pattern (Checkout Payment)

i. Function or Software Component Affected

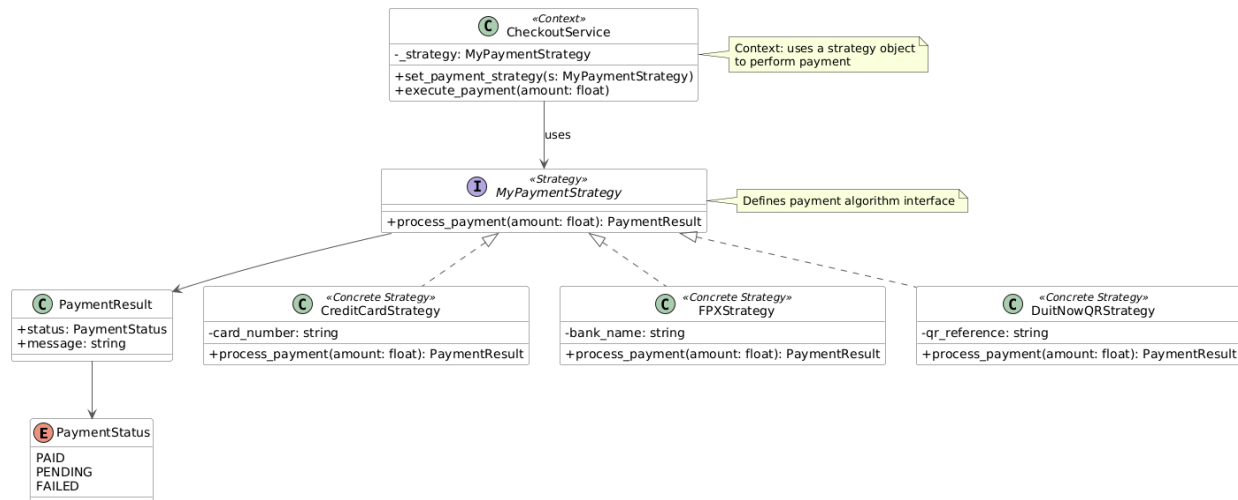
The Strategy Pattern is applied to the Checkout Payment module, which must support multiple payment methods — Credit Card, FPX Online Banking, and DuitNow QR — without hardcoding conditional logic into the core checkout flow.

ii. Description of Workflow / Data Flow

When a customer reaches the TechVault checkout page, they select a payment method from the available options. The selected method is wrapped in a concrete PaymentStrategy object and injected into the CheckoutService via `set_payment_strategy()`. When the customer clicks Pay, the CheckoutService calls `execute_payment(amount)` — it does not know or care which payment method was selected. The actual payment logic resides entirely within the selected concrete strategy class. This design allows TechVault to add new local payment options such as Touch 'n Go eWallet or GrabPay by simply creating a new strategy class, without modifying the CheckoutService or any other existing code

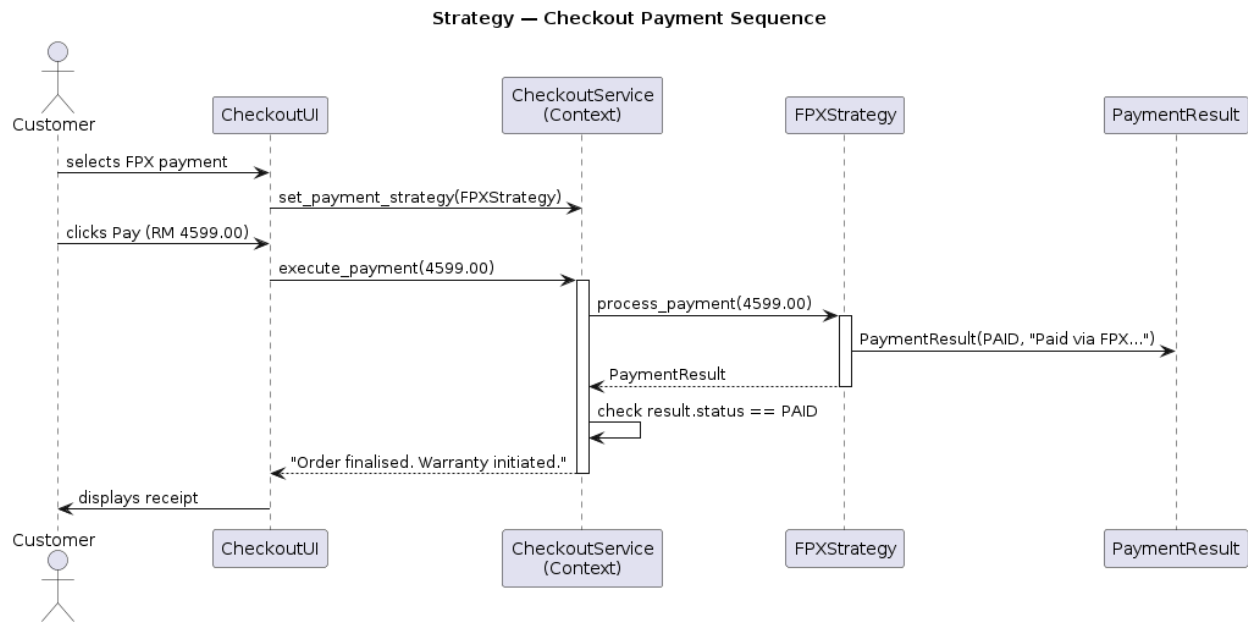
iii. Class Diagram

The class diagram features the MyPaymentStrategy interface prominently in the center, with a single `process_payment(amount)` method which returns a PaymentResult. Three concrete strategy classes implement this interface: CreditCardStrategy, FPXStrategy, and DuitNowQRStrategy. Each of these concrete strategies includes attributes specific to their respective payment types, for example, `cardNumber`, `bankName`, `QRReference`. The CheckoutService serves as the context - it contains an MyPaymentStrategy reference through composition, and therefore may interact with any of the strategies even though it does not know which one it is. The PaymentResult class includes a PaymentStatus enum (PAID, PENDING, FAILED) and message string. Adding new payment types can be accomplished simply by creating new classes implementing MyPaymentStrategy without modifying the CheckoutService.



iv. Sequence Diagram

The sequence diagram represents a customer paying using the FPX strategy during checkout. The UI first calls `set_payment_strategy(FPXStrategy)` on the `CheckoutService`, passing it the `FPXStrategy` reference. The customer then clicks the 'Pay' button, triggering the UI to call `execute_payment(4599.00)` on `CheckoutService`. The `CheckoutService` delegates the call to `FPXStrategy.process_payment(4599.00)`, which creates a new `PaymentResult` instance with a status of `PAID`, and returns that object to the `CheckoutService`. Upon receiving the `PaymentResult` from `FPXStrategy`, the `CheckoutService` checks the `PaymentResult` status and, if successful, confirms the order and initiates warranty registration. Finally, the UI displays the invoice to the customer. At no point in time does `CheckoutService` ever have any payment type specific logic.



v. Sample Potential Code

```

from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum

class PaymentStatus(Enum):
    PAID = 'Paid'
    PENDING = 'Pending'
    FAILED = 'Failed'

@dataclass
class PaymentResult:
    status: PaymentStatus
    message: str

# Strategy Interface
class IPaymentStrategy(ABC):
    @abstractmethod
    def process_payment(self, amount: float) -> PaymentResult: pass
  
```

```

# Concrete Strategy: Credit Card
class CreditCardStrategy(IPaymentStrategy):
    def __init__(self, card_number: str):
        self.card_number = card_number

    def process_payment(self, amount: float) -> PaymentResult:
        return PaymentResult(PaymentStatus.PAID,
                               f'Paid RM{amount:.2f} via card ending
{self.card_number[-4:]}')

# Concrete Strategy: FPX Online Banking
class FPXStrategy(IPaymentStrategy):
    def __init__(self, bank_name: str):
        self.bank_name = bank_name

    def process_payment(self, amount: float) -> PaymentResult:
        return PaymentResult(PaymentStatus.PAID,
                               f'Paid RM{amount:.2f} via FPX ({self.bank_name})')

# Concrete Strategy: DuitNow QR
class DuitNowQRStrategy(IPaymentStrategy):
    def __init__(self, qr_reference: str):
        self.qr_reference = qr_reference

    def process_payment(self, amount: float) -> PaymentResult:
        return PaymentResult(PaymentStatus.PAID,
                               f'Paid RM{amount:.2f} via DuitNow QR (Ref:
{self.qr_reference})')

# Context: CheckoutService - unaware of which strategy is selected
class CheckoutService:
    def __init__(self): self._strategy = None

    def set_payment_strategy(self, s: IPaymentStrategy): self._strategy =
s

    def execute_payment(self, amount: float):
        if not self._strategy:

```

```

        raise ValueError('No payment strategy configured.')

    result = self._strategy.process_payment(amount)
    print(result.message)

    if result.status == PaymentStatus.PAID:
        print('Order finalized. Warranty registration initiated.')

# Example: Customer selects FPX at checkout
checkout = CheckoutService()

checkout.set_payment_strategy(FPXStrategy('Maybank2u'))

checkout.execute_payment(4599.00)

```

The code implements the Strategy Pattern for the TechVault checkout payment module using Python. It begins by defining a `PaymentStatus` enum with three possible values — `PAID`, `PENDING`, and `FAILED` — and a `PaymentResult` dataclass that carries a `PaymentStatus` value and a descriptive message string, serving as the standardized return object for every payment operation. The `IPaymentStrategy` abstract base class defines the contract that all payment methods must follow, declaring one abstract method `process_payment(amount)` that every concrete strategy must implement. Three concrete strategy classes implement this interface — `CreditCardStrategy` stores a card number and returns a result confirming payment via the last four digits of the card, `FPXStrategy` stores a bank name and returns a result confirming payment via FPX online banking, and `DuitNowQRStrategy` stores a QR reference code and returns a result confirming payment via DuitNow QR — each encapsulating its own payment logic entirely within its `process_payment()` method. The `CheckoutService` class acts as the context, initializing `_strategy` as `None` and exposing two methods — `set_payment_strategy()` which injects whichever concrete strategy the customer selects at runtime, and `execute_payment()` which first guards against a missing strategy by raising a `ValueError`, then delegates the entire payment operation to `self._strategy.process_payment(amount)` without containing any payment-specific logic itself. Upon receiving the `PaymentResult`, `execute_payment()` prints the result message and, if the status is `PAID`, confirms the order and initiates warranty registration. In the client code at the bottom, a `CheckoutService` is instantiated, `FPXStrategy` with `Maybank2u` is injected as the selected payment method, and `execute_payment(4599.00)` is called — demonstrating that the checkout flow remains identical regardless of which payment strategy is chosen, and that new payment methods can be added in the future simply by creating a new class implementing `IPaymentStrategy` without modifying `CheckoutService` at all.

vi. Benefits

The Strategy Pattern eliminates cascading if-else or switch-case blocks from the checkout logic by encapsulating each payment method in its own class. New payment methods can be introduced without any modification to the `CheckoutService`, directly adhering to the Open/Closed Principle. This is especially valuable for TechVault where Malaysian consumer preference requires support for multiple local payment options.

vii. Limitations

The pattern increases the total number of classes in the system. For a platform with only one payment method, this overhead may be unnecessary. However, for TechVault's multi-method payment requirement, the structural clarity and extensibility provided by the Strategy Pattern far outweighs the overhead of additional classes.

c. Design Pattern 3: State Pattern (Order Lifecycle)

i. Function or Software Component Affected

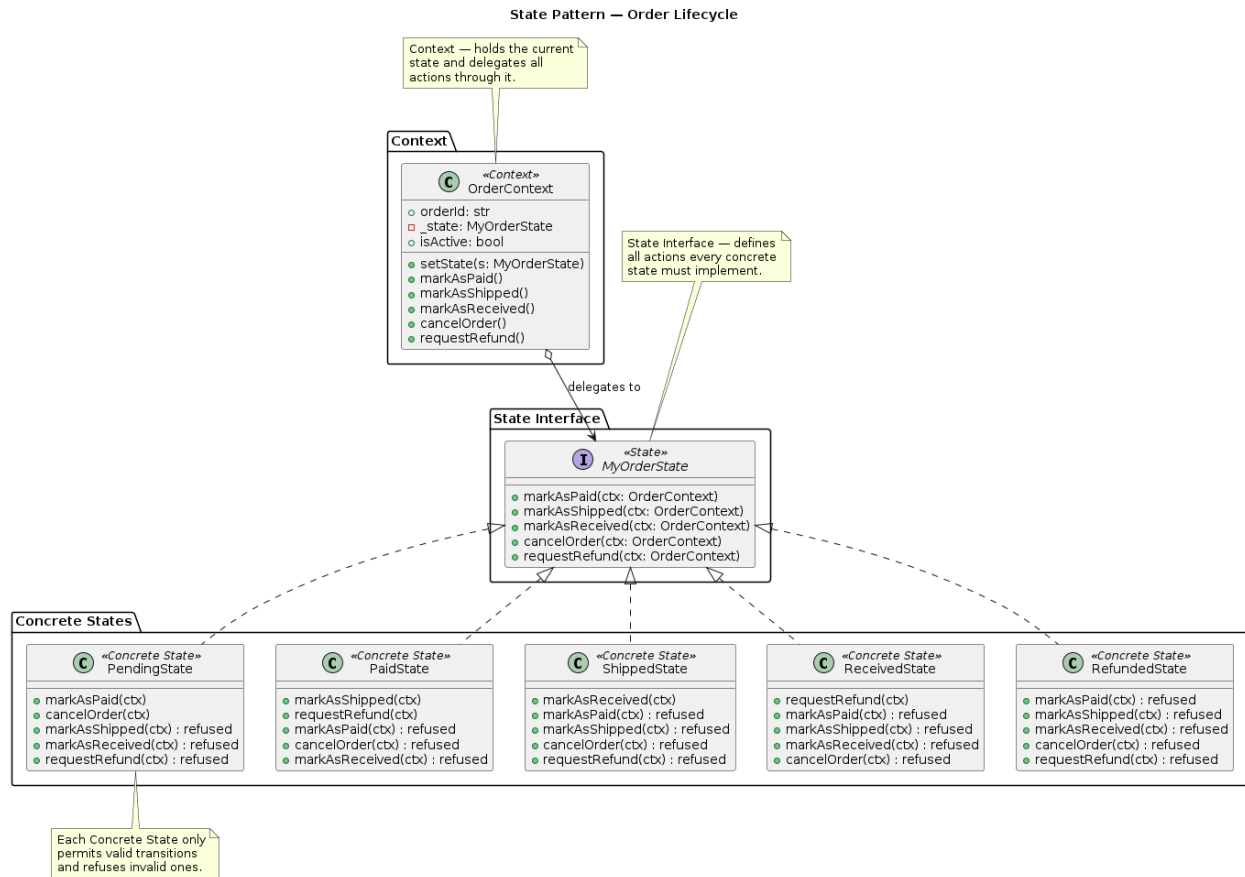
The State Pattern is applied to the Order Lifecycle module, which manages the valid transitions of an electronics order from Pending through to Received or Refunded.

ii. Description of Workflow / Data Flow

When a customer completes checkout, the OrderContext is initialized in PendingState. Upon payment confirmation (via the Strategy Pattern), the context transitions to PaidState. The vendor then dispatches the device, transitioning the order to ShippedState. Upon customer delivery confirmation, the state moves to ReceivedState. At any valid point, an eligible order may transition to RefundedState. Each state class defines which transitions are valid from that state, and all invalid transitions such as attempting to mark a Pending order as Shipped are explicitly rejected with a clear error message. This ensures that the order lifecycle remains consistent and auditable, which is critical for high-value electronics transactions.

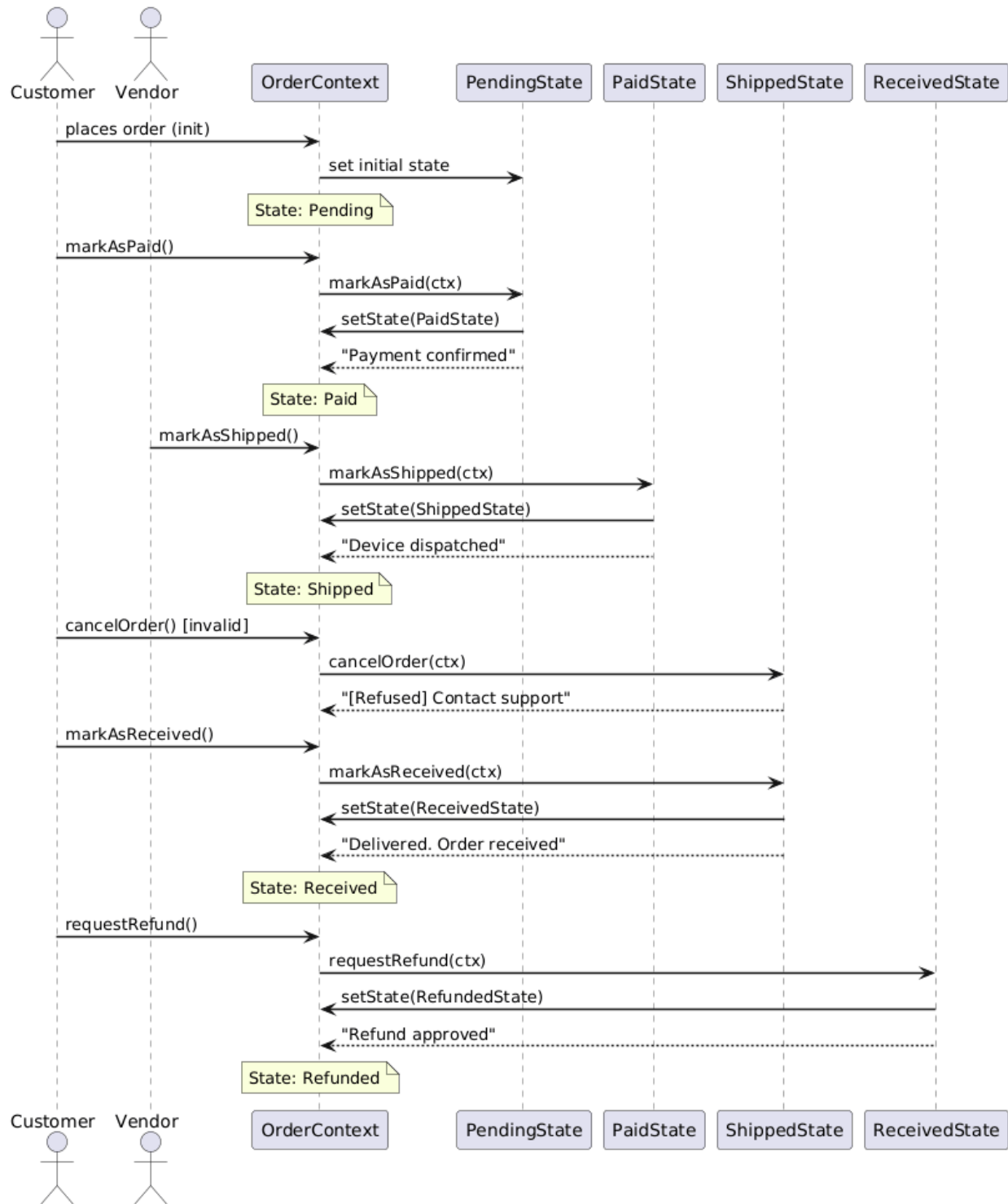
iii. Class Diagram

MyOrderState interface in Class Diagram has five methods - markAsPaid(), markAsShipped(), markAsReceived(), cancelOrder() and requestRefund() that implement by all concrete states. The five state classes PendingState, PaidState, ShippedState, ReceivedState and RefundedState implement the MyOrderState interface and allow valid state transitions but refuse to allow invalid transitions explicitly. The OrderContext class contains the current state as its private attribute (_state) and passes any method calls through to this state via setState(). Change in the context's order behavior occurs automatically as the state of the context is changed, with no conditional logic within the OrderContext class itself.



iv. Sequence Diagram

The entire realistic order life-cycle can be traced in the Sequence Diagram. A customer places an order (initializing OrderContext as PendingState), then when payment is received, PendingState.markAsPaid() changes the context to PaidState. At that point, the vendor ships the order, which causes PaidState.markAsShipped() to change the context from PaidState to ShippedState. Attempting to cancel an order at this point will return a refusal message and not change the state in ShippedState. After delivery is confirmed, ShippedState.markAsReceived() will change the order state to ReceivedState. All transitions are enforced within the state class; the context simply delegates.



v. Sample Potential Code

vi. Benefits

The State Pattern eliminates complex nested conditionals from the order management code by encapsulating all state-specific behavior within individual state classes. Invalid transitions are automatically rejected at the class level, providing a self-enforcing workflow that is critical for high-value electronics transactions. The pattern also integrates naturally with Design Pattern 2 (Strategy), as the transition to PaidState signals that the selected payment strategy executed successfully.

vii. Limitations

The pattern results in a larger number of classes proportional to the number of states. For TechVault's five-state order lifecycle, this overhead is well-justified. However, for systems with very few states (two or three), the State Pattern may introduce more structural complexity than simpler conditional logic would require.

d. Design Pattern 4: Factory Method (Role-Based Dashboard Routing)

i. Function or Software Component Affected

The Factory Method Pattern is applied to the web page rendering module, specifically for role-based dashboard routing between the Customer dashboard and the Admin dashboard.

ii. Description of Workflow / Data Flow

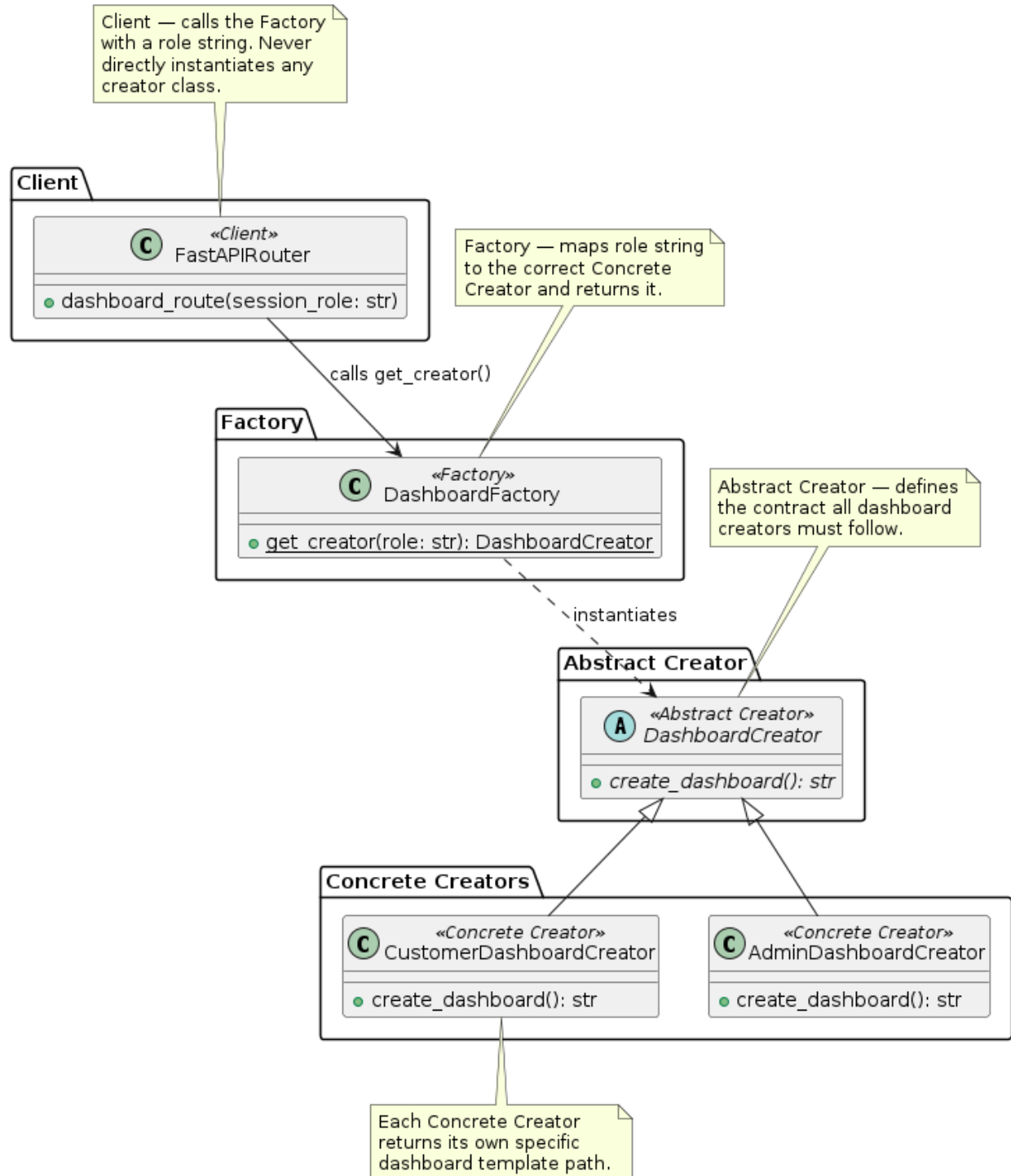
When a user accesses the /dashboard route after signing in, the FastAPI Router reads the user's session role (Customer or Admin) and delegates dashboard instantiation to the PageFactory. The factory creates the appropriate DashboardPage concrete object based on the role, and the returned object provides the correct HTML template path for rendering. This keeps the routing logic entirely decoupled from specific dashboard implementations. Adding a new role — such as Technician or Analyst — requires only creating a new concrete DashboardCreator and adding a branch to the factory, without any modification to the Router.

iii. Class Diagram

This class diagram shows the abstract class called DashboardCreator that has a method called create_dashboard() which will return a string representing the path of a template. There are two concrete implementations of DashboardCreator: CustomerDashboardCreator, which provides the path to templates/user_dashboard.html, and AdminDashboardCreator, which provides the path to templates/admin_dashboard.html. The DashboardFactory has a single static method, get_creator(role), that maps a role to a specific DashboardCreator subclass. When a request is made to the /dashboard resource through a FastAPI router, it will call DashboardFactory.get_creator() with the appropriate role and use the returned

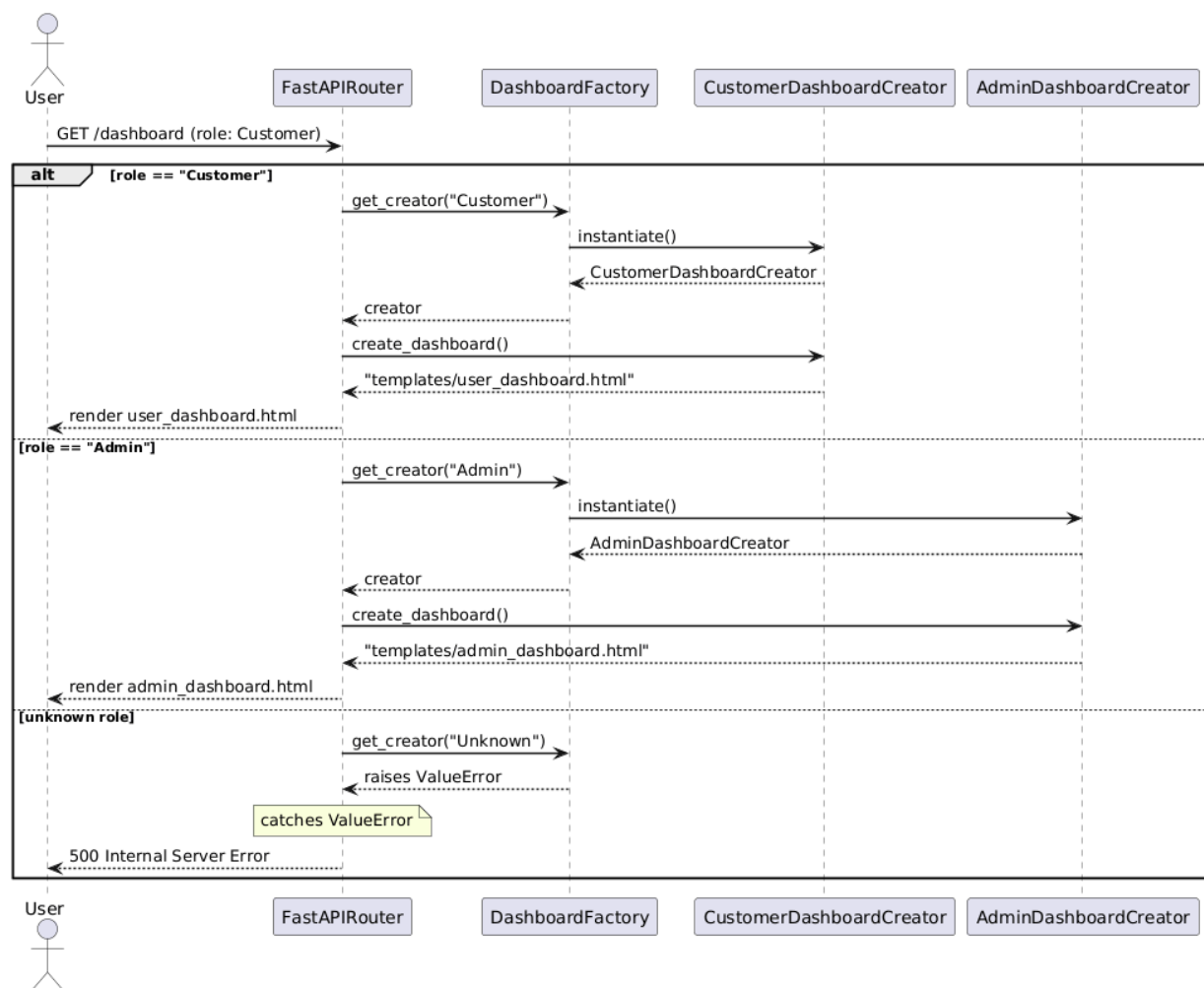
DashboardCreator to render the appropriate dashboard. The router will never directly instantiate an instance of either DashboardCreator, keeping the router decoupled from the specific DashboardCreator implementations.

Factory Method Pattern — Role-Based Dashboard Routing



iv. Sequence Diagram

The sequence diagram depicts the expected behavior based on three branching scenarios when a user requests the /dashboard resource. If the user's session role is Customer, the router will call `DashboardFactory.get_creator("Customer")` to instantiate a `CustomerDashboardCreator` and return it to the router. The router will then call `CustomerDashboardCreator.create_dashboard()` to get the path of the customer dashboard template and render it. The Admin scenario works in exactly the same way but will use the admin dashboard template. In the last scenario, when an invalid role is assigned to the user, the `DashboardFactory` raises a `ValueError` and the router returns a 403 Forbidden response. Regardless of the path being followed through the sequence of events or of the type of `DashboardCreator` being used to create a dashboard, the routing logic remains the same, but only the output from the `DashboardFactory` will differ.



v. Sample Potential Code

```
from abc import ABC, abstractmethod

# Abstract Creator
class DashboardCreator(ABC):
    @abstractmethod
    def create_dashboard(self) -> str: pass

# Concrete Creator: Customer
class CustomerDashboardCreator(DashboardCreator):
    def create_dashboard(self) -> str:
        return 'templates/user_dashboard.html'

# Concrete Creator: Admin / Vendor
class AdminDashboardCreator(DashboardCreator):
    def create_dashboard(self) -> str:
        return 'templates/admin_dashboard.html'

# Factory - maps role string to correct creator
class DashboardFactory:
    @staticmethod
    def get_creator(role: str) -> DashboardCreator:
        creators = {
            'Customer': CustomerDashboardCreator,
            'Admin': AdminDashboardCreator,
        }
        if role not in creators:
            raise ValueError(f'Unknown role: {role}')
        return creators[role]()
```

```
# FastAPI Route Handler (simulated)
def dashboard_route(session_role: str):
    creator = DashboardFactory.get_creator(session_role)
    template = creator.create_dashboard()
    print(f'Rendering: {template}')

# Demo
dashboard_route('Customer') # Rendering: templates/user_dashboard.html
dashboard_route('Admin')    # Rendering: templates/admin_dashboard.html
```

The code implements the Factory Method Pattern for the TechVault role-based dashboard routing module using Python. It begins by defining the abstract base class `DashboardCreator` which inherits from `ABC` and declares one abstract method `create_dashboard()` returning a string — this is the contract that every concrete creator must fulfill, and because it is abstract, `DashboardCreator` cannot be instantiated directly. Two concrete creator classes extend this abstract class — `CustomerDashboardCreator` implements `create_dashboard()` and returns the string `'templates/user_dashboard.html'`, while `AdminDashboardCreator` implements the same method and returns `'templates/admin_dashboard.html'` — each encapsulating its own specific template path without any knowledge of the other. The `DashboardFactory` class contains a single static method `get_creator(role)` which internally defines a dictionary named `creators` that maps the role strings `'Customer'` and `'Admin'` to their respective creator classes — note that the dictionary stores the class references themselves, not instances, which is why `creators[role]()` includes parentheses at the end to instantiate the correct class at the point of retrieval. If the provided role string is not found in the dictionary, a `ValueError` is raised immediately with a descriptive message, preventing the application from proceeding with an unrecognized role. The `dashboard_route()` function simulates a FastAPI route handler — it calls `DashboardFactory.get_creator(session_role)` to retrieve the correct creator, then calls `create_dashboard()` on it to get the template path, and prints the result. In the demo at the bottom, calling `dashboard_route('Customer')` renders the customer dashboard template and calling `dashboard_route('Admin')` renders the admin dashboard template — demonstrating that the routing function itself never changes and never directly references either concrete creator class, meaning new user roles can be added in the future simply by creating a new concrete creator class and adding one entry to the factory dictionary, with zero modification to any existing code.

vi. Benefits

By offloading dashboard instantiation to the Factory, the main application router remains clean and readable regardless of how many user roles the system grows to support. Adding a new dashboard type requires only a new concrete creator class — the router code requires zero modification, directly adhering to the Open/Closed Principle.

vii. Limitations

Since instantiation occurs in the factory rather than the router, error handling for invalid or missing role values must be implemented robustly in the factory itself. A missing role check would cause the application to fail silently at the factory stage rather than at the more visible routing stage.

e. Design Pattern 5: Observer Pattern (Notification System)

i. Function or Software Component Affected

The Observer Pattern is applied to the Notification Module, which dispatches automated status updates to customers, refreshes the admin dashboard, and logs audit records whenever an order lifecycle state changes.

ii. Description of Workflow / Data Flow

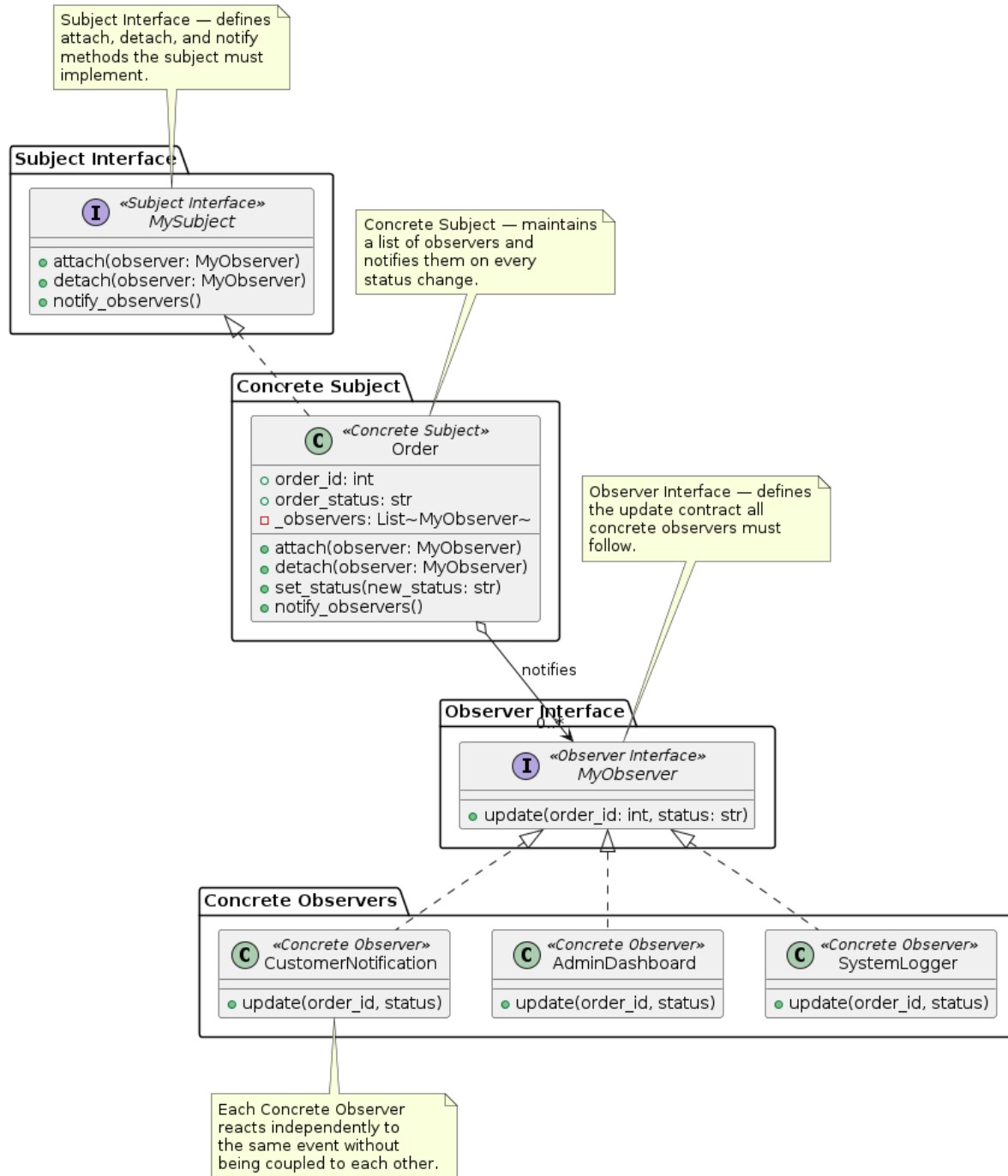
When the admin updates an order status — for example, transitioning it to Shipped — the Order object (the Subject) triggers a notification process. Every registered observer performs its own independent action: the CustomerNotification observer sends the customer an order update; the AdminDashboard observer refreshes the pending orders display; and the SystemLogger observer records the state change for audit purposes. The Order class is completely unaware of the number or type of observers; it simply calls `notify_observers()` and each registered listener handles the event according to its own logic. Adding or removing observers requires no changes to the Order class.

iii. Class Diagram

The class diagram illustrates the Observer Pattern applied to the TechVault notification system, organized into four clearly labeled structural packages. At the top left, the Subject Interface package contains the `MySubject` interface stereotyped as «Subject Interface», which defines three methods — `attach()`, `detach()`, and `notify_observers()` — representing the contract that any observable subject must fulfill for managing its list of listeners. Below it, the Concrete Subject package contains the `Order` class stereotyped as «Concrete Subject», which implements `MySubject` and holds three attributes — `order_id`, `order_status`, and a private `_observers` list typed as `List~MyObserver~` — along with four methods: `attach()` and `detach()` for managing the observer list, `set_status()` for updating the order's state, and `notify_observers()` for broadcasting the change to all registered listeners. The `Order` class is connected to the Observer Interface package below it via a composition arrow labeled "notifies", pointing to the `MyObserver` interface stereotyped as «Observer Interface», which defines a single `update(order_id, status)` method as the contract all concrete observers must implement. At the bottom, the Concrete Observers package contains three classes — `CustomerNotification`, `AdminDashboard`, and `SystemLogger` — all stereotyped as «Concrete Observer» and connected to `MyObserver` via dashed realization arrows, each

implementing the `update()` method to independently handle the broadcasted event in their own way, whether that is alerting the customer, refreshing the vendor dashboard, or recording an audit log entry. The note at the bottom reinforces the key design benefit of this pattern — that each concrete observer reacts to the same event independently without being coupled to one another, meaning observers can be freely added or removed without any modification to the Order class itself.

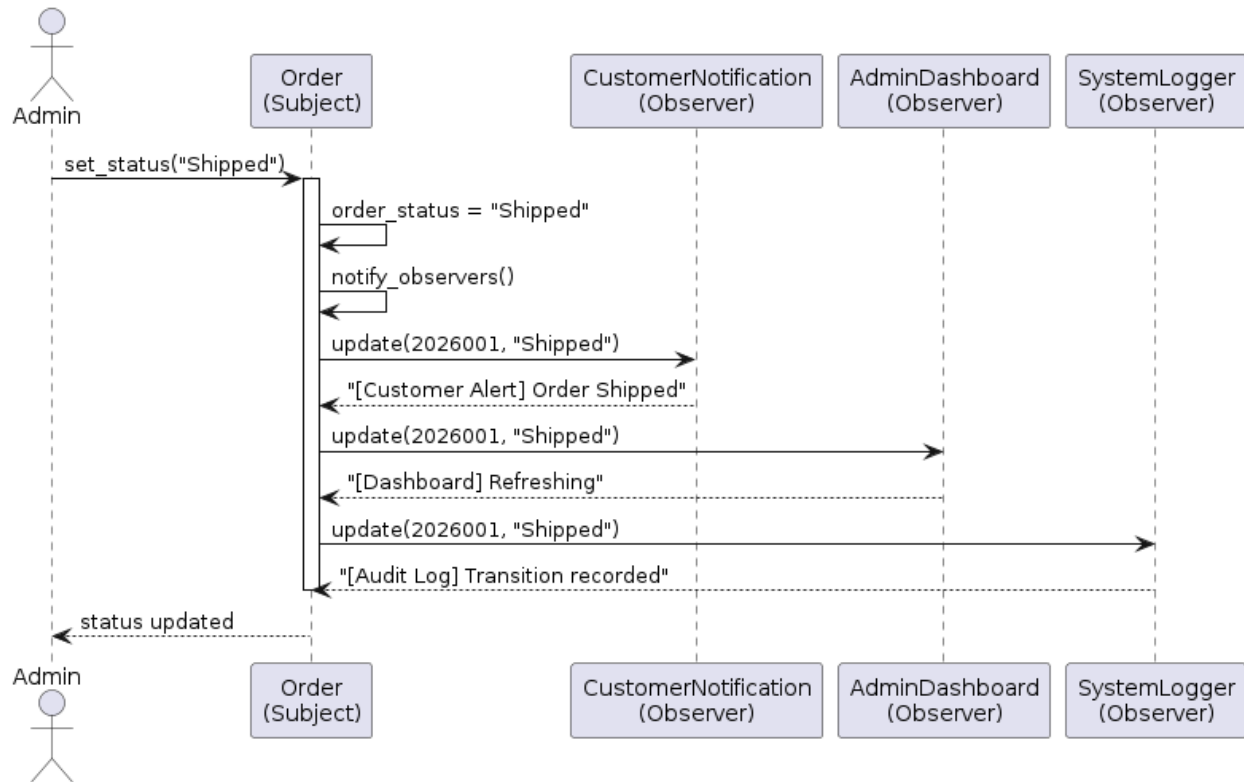
Observer Pattern — Notification System



iv. Sequence Diagram

The sequence diagram illustrates the Observer Pattern applied to the TechVault notification system, tracing the complete flow of events that occur when an admin updates an order status to "Shipped". The diagram involves five participants — the Admin actor, the Order (Subject), and three observers: CustomerNotification (Observer), AdminDashboard (Observer), and SystemLogger (Observer). The flow begins with the Admin sending a `set_status("Shipped")` call to the Order object, which triggers two immediate internal actions shown by the self-referencing arrows — first updating its own `order_status` attribute to "Shipped", then calling `notify_observers()` to begin broadcasting the change to all registered listeners. The Order then sequentially calls `update(2026001, "Shipped")` on each of the three registered observers one by one — CustomerNotification receives the call first and responds with "[Customer Alert] Order Shipped", followed by AdminDashboard which responds with "[Dashboard] Refreshing", and finally SystemLogger which responds with "[Audit Log] Transition recorded". Each observer handles the notification independently and returns its own response without any awareness of the other observers, demonstrating the decoupled nature of the pattern. Once all three observers have been notified, the Order returns a "status updated" confirmation back to the Admin, completing the entire notification chain. The key design behavior shown in this diagram is that a single `set_status()` call from the admin automatically triggers three entirely separate downstream actions across different system components, all coordinated by the Order subject without the admin or any observer needing to be aware of each other.

Observer — Notification Sequence



v. Sample Potential Code

```

from abc import ABC, abstractmethod
from typing import List

# Observer interface
class MyObserver(ABC):
    @abstractmethod
    def update(self, order_id: int, order_status: str) -> None: pass

# Subject interface
class MySubject(ABC):
    @abstractmethod
    def attach(self, observer: MyObserver) -> None: pass
    @abstractmethod
    def detach(self, observer: MyObserver) -> None: pass
  
```

```

    @abstractmethod
    def notify_observers(self) -> None: pass

# Concrete Subject: Order
class Order(MySubject):
    def __init__(self, order_id: int):
        self.order_id = order_id
        self.order_status = 'Pending'
        self._observers: List[MyObserver] = []

    def attach(self, observer: MyObserver):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer: MyObserver):
        if observer in self._observers:
            self._observers.remove(observer)

    def set_status(self, new_status: str):
        self.order_status = new_status
        print(f'Order #{self.order_id} status changed to: {self.order_status}')
        self.notify_observers()

    def notify_observers(self):
        for observer in self._observers:
            observer.update(self.order_id, self.order_status)

# Concrete Observers
class CustomerNotification(IObserver):
    def update(self, order_id, order_status):
        print(f' [Customer Alert] Order #{order_id} is now: {order_status}.')

```

```

class AdminDashboard(IObserver):
    def update(self, order_id, order_status):
        print(f' [Dashboard] Order #{order_id} updated to {order_status}. Refreshing.')

class SystemLogger(MyObserver):
    def update(self, order_id, order_status):
        print(f' [Audit Log] Order #{order_id} transitioned to {order_status}.')

# Demo
order = Order(order_id=2026001)
for obs in [CustomerNotification(), AdminDashboard(), SystemLogger()]:
    order.attach(obs)

order.set_status('Paid')
order.set_status('Shipped')

```

The code implements the Observer Pattern for the TechVault notification system using Python. It begins by defining two abstract base classes — `MyObserver`, which declares one abstract method `update(order_id, order_status)` as the contract all concrete observers must implement, and `MySubject`, which declares three abstract methods — `attach()`, `detach()`, and `notify_observers()` — as the contract the concrete subject must fulfill for managing its observer list. The `Order` class acts as the concrete subject by extending `MySubject` and implementing all three required methods — its constructor initializes `order_id`, sets `order_status` to 'Pending' by default, and creates an empty `_observers` list typed as `List[MyObserver]` to store all registered listeners. The `attach()` method adds an observer to the list only if it is not already present, preventing duplicate registrations, while `detach()` removes an observer from the list only if it exists, preventing errors from removing unregistered observers. The `set_status()` method is the core trigger of the pattern — it updates the internal `order_status` attribute, prints a status change message, and immediately calls `notify_observers()`, which iterates through every observer in `_observers` and calls `update(order_id, order_status)` on each one, passing the current order ID and new status. Three concrete observer classes implement `MyObserver` — `CustomerNotification` prints a customer-facing alert, `AdminDashboard` prints a dashboard refresh notification, and `SystemLogger` prints an audit log entry — each handling the same broadcasted event independently in their own way. In the demo at the bottom, an `Order` is created with ID 2026001, and all three concrete observer instances are registered using a `for` loop that calls

`order.attach()` on each one. Calling `order.set_status('Paid')` then `order.set_status('Shipped')` triggers two full notification cycles, each time printing the status change and then sequentially notifying all three observers — demonstrating that the `Order` class remains completely unaware of what each observer does with the notification, and that new observers can be added at any time simply by creating a new class implementing `MyObserver` and attaching it to the order, with zero modification to the `Order` class itself.

vi. Benefits

The Observer Pattern provides high flexibility by allowing multiple system components to react to a single event without being directly coupled to each other. This is especially important for TechVault where a single order status change must simultaneously notify the customer, update the vendor dashboard, and log to the audit system. Adding a new observer — such as a `WarrantyActivationObserver` when the order reaches `Received` — requires no modification to the `Order` class.

vii. Limitations

Performance overhead may increase as the number of registered observers grows, since every state change triggers updates to all listeners. Additionally, the indirect communication model makes debugging more complex, as it can be less immediately obvious which observer is responsible for a specific side effect. These risks are managed by keeping observer implementations small and focused, consistent with the Single Responsibility Principle.

4. Conclusion and Suggestion

5.1 Conclusion

This document has presented the complete software design and architectural specification of TechVault, a single-vendor Consumer Electronics E-Commerce Platform built to address the operational challenges faced by independent electronics retailers. The system replaces fragmented, manual sales and inventory management processes with a centralized, automated web application that handles the full retail lifecycle from specification-driven product discovery through to order fulfillment, warranty registration, and after-sales support.

The architecture is governed by five core software design concepts — Modularity, Abstraction, Encapsulation, Refactoring, and Separation of Concerns — and rigorously applies all five SOLID design principles to ensure low coupling and high cohesion across the codebase. The five proposed design patterns collectively address the system's key structural and behavioral challenges: the Facade Pattern simplifies the admin dashboard interface; the Strategy Pattern enables flexible, extensible payment processing; the State Pattern enforces a consistent and auditable order lifecycle; the Factory Method Pattern cleanly handles role-based dashboard routing; and the Observer Pattern decouples order state changes from their downstream notification effects. Together, these patterns produce a maintainable, extensible architecture that can accommodate future business requirements without requiring significant structural changes to the existing codebase.

5.2 Suggestion

Several enhancements are proposed for future development iterations of TechVault. First, integrating a live payment gateway supporting FPX, DuitNow, and major credit card networks would replace the current mock payment simulation and enable real commercial transactions, requiring PCI-DSS compliance planning. Second, the product catalog module could be enhanced with a structured side-by-side specification comparison engine, allowing customers to directly compare two or more devices across a standardized attribute grid — directly addressing the specification-driven purchasing behavior documented in the literature review. Third, the warranty module could be extended to support automated expiry notifications, alerting customers when their registered product warranty is approaching its expiry date through the Observer Pattern. Fourth, as the product catalog and order volume scale, the database backend should be migrated from SQLite to a production-grade relational database such as MySQL or PostgreSQL for improved concurrency and reliability. Finally, a comprehensive testing strategy covering unit tests for all service layer classes, integration tests for the database access layer, and user acceptance testing of the full checkout flow should be conducted before any production launch to ensure system reliability, data integrity, and security at scale.

5. Bibliography/References

- [1] K. R. Ranjan, "The impact of electronic commerce on business organization," *Research Journal of Arts, Management & Social Sciences*, vol. 22, no. 11, pp. 80–89, 2022.
- [2] W. M. Lim, C. S. Phang, and A. Lim, "Consumer behaviour and online purchase intention in the electronics sector," *Journal of Retailing and Consumer Services*, vol. 70, pp. 103–115, 2023.
- [3] D. Castaño-Herrera, J. Gomez, L. Garrido, L. Tapiador, and J. M. Vassallo, "How does e-commerce impact shopping mobility behavior?" *Transportation*, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s11116-025-10658-x>